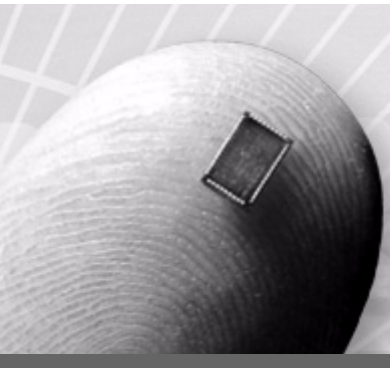
ON Semiconductor®



Introduction to BELASIGNA® 300 Programming

M-20742-004

ON Semiconductor

November 2009

This document may not be redistributed or transmitted in any form.

Copyright © 2009 Semiconductor Components Industries, LLC d/b/a ON Semiconductor.

ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA and SIGNAKLARA are either trademarks or registered trademarks of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.

ON Semiconductor reserves the right to make changes without further notice to any products herein. ON Semiconductor makes no warranty, representation or guarantee regarding its products including suitability for any particular purpose, nor does ON Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in ON Semiconductor data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. ON Semiconductor does not convey any license under its patent rights nor the rights of others. ON Semiconductor products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the ON Semiconductor product could create a situation where personal injury or death may occur, unless otherwise expressly agreed to in writing by ON Semiconductor and BUYER. Should Buyer purchase or use ON Semiconductor products for any such unintended or unauthorized application, Buyer shall indemnify and hold ON Semiconductor and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that ON Semiconductor was negligent regarding the design or manufacture of the part. ON Semiconductor is an Equal Opportunity/Affirmative Action Employer. Literature is subject to all applicable copyright laws and is not for resale in any manner.

To maintain the quality of our publications, we request your comments on the accuracy, clarity, organization, and value of this document.

Address correspondence to:

Americas and Asia
ON Semiconductor
200-611 Kumpf Drive
Waterloo, Ontario, Canada N2V 1K8
Tel: +1 519 884 9696

Europe, Middle East and Africa
ON Semiconductor
Champs-Montants 12a
2074 Marin, Switzerland
Tel: +41 32 755 74 30

Website
www.onsemi.com

E-mail
dsp.support@onsemi.com

TABLE OF CONTENTS

1. Introduction 1

1.1 Purpose 1
1.2 Intended Audience 1
1.3 Conventions 1
1.4 Manual Organization 2
1.5 Further Reading 2

2. Architecture Overview 3

2.1 Introduction 3
2.2 CFX Digital Signal Processor 4
2.2.1 CFX Memory 4
2.3 HEAR Configurable Accelerator 5
2.3.1 Shared Memory 5
2.4 FIFO Controller 6
2.5 Input/Output Controller 6
2.6 Input Stage 7
2.7 Output Stage 7
2.8 PCM Interface 7
2.9 Peripherals 7
2.10 External Interfaces 8
2.11 Clocking 9
2.12 Power Management 9
2.13 Firmware Components 9

3. Design Information 11

3.1 Introduction 11
3.2 Then and Now 11
3.3 Programming a Dual-Core System 11
3.3.1 Partitioning Algorithms 12
3.4 Data Flow 13

4. Introduction to Sample Code 17

4.1 Introduction 17
4.2 Accessing the Sample Code 17
4.3 Accessing the Firmware Libraries 19
4.4 Code Structure 19
4.5 File Structure 21
4.5.1 CFX Source Code File Structure 21
4.5.2 HCT Input File Structure 22
4.6 Application Initialization 22
4.7 Modifying the Sample Code Applications—An Example . . . 23

5. Porting Guide 27

5.1 Introduction 27
5.2 Development Tools 27
5.2.1 Toolchain 27
5.2.2 Integrated Development Environment 28
5.2.3 Chip IDs 28
5.3 CFX Programming 28
5.3.1 Instruction Types 29
5.3.2 Syntax Basics 30
5.3.3 Delay Slots and Synthetic Instructions 30
5.3.4 Registers 31
5.3.5 Loops 32
5.3.6 Stack Usage 34
5.3.6.1 Stack Initialization 34
5.3.7 Control Flow Instructions 35
5.3.8 Arithmetic Instructions 36
5.3.9 Multiplication Instructions 40
5.3.9.1 Multiplication Instruction Mappings 41
5.3.9.2 Multiplication Restrictions 43
5.3.10 Comparison Instructions and Condition Codes 44
5.3.11 Data Movement Instructions 46
5.3.11.1 Other Data Movement Instructions 48

5.3.11.2 Address Arithmetic and Addressing Modes 49
5.3.12 Rounding and Saturation 50
5.3.13 Interrupt Service Routines 50
5.4 FIFO Controller and IOC Configuration 51
5.4.1 FIFO Controller Configuration 51
5.4.2 IOC Configuration 52
5.5 HEAR Configuration 53
5.5.1 Application Design Considerations 53
5.5.2 Configuration Using the HCT 54
5.5.3 Implementing the WOLA Filterbank 55
5.5.3.1 Defining Constants. 55
5.5.3.2 Configuring the FIFO Controller 56
5.5.3.3 Configuring the WOLA Module 57
5.5.3.4 Analysis, Gain and Synthesis Functions. 58
5.5.3.5 CFX-HEAR Communication 59
5.6 Other System Configuration Tasks 60
5.6.1 Configuration of Analog Blocks 60
5.6.2 Interrupt Controller Configuration 61
5.6.3 Clock Configuration 62
5.6.4 Shared Memory Access 63
5.6.5 Low-power Mode 63
5.7 Sample Code Structure Comparison 64
5.7.1 Source File and Code Structure 64
5.7.2 File-by-file Comparison 67
A. Glossary *69*

This page left blank for printing purposes.

1 Introduction

1.1 Purpose

The intention of this document is to help you start developing applications on BELASIGNA® 300 as quickly as possible. Whether you are new to the DSP technology of ON Semiconductor or experienced, you can benefit from the general design information in this manual. If you have used BelaSigna 250 or BelaSigna 200, this manual will help you transition to BelaSigna 300 by describing changes to the hardware architecture and programming methods, and providing a comparison of sample code to show differences. If you are a new user, you can still benefit from the use of sample code to learn about writing programs for BelaSigna 300.

This manual describes some of the significant improvements in the chip that, when properly exploited, can provide vastly improved performance and open up a number of possibilities for implementing new features in your algorithms. Key improvements include:

- A more capable, dual-MAC 24-bit DSP core, the CFX
- A new and more capable signal processing engine, the HEAR Configurable Accelerator
- An enhanced data flow manager, the Input/Output Controller (IOC), that provides improved data routing between components in the system
- Improved, more flexible FIFOs for efficient data processing
- Improved efficiency (MIPS/mW)
- Improved system integration for RF applications through a flexible IOC and PCM interface enhancements
- 130 nm semiconductor technology, providing advanced, complex functions at lower power consumption and smaller physical die size than the previous 180 nm technology
- High fidelity
- Enhanced peripherals, such as a CRC generator, and a security module with high-grade encryption capability

This manual is merely the introduction to programming BelaSigna 300. For details, see the other manuals listed in Section 1.5, “Further Reading” on page 2.

1.2 Intended Audience

This manual is aimed at software development professionals or signal processing engineers who are new to BelaSigna 300, whether they have previous experience with BelaSigna 250 or BelaSigna 200 or not. We assume readers have experience developing applications in assembly language, and are somewhat familiar with digital signal processing, embedded software development, or both.

1.3 Conventions

This manual contains assembly language programming examples for the RCore DSP and the CFX DSP. All of these examples use a fixed-width font to differentiate them from the rest of this manual's content. The CFX examples are differentiated from the RCore examples with the use of a bold font, as shown in the example below.

```
// This is an example of RCore assembly code.
LDI AH, 0x7fff

// This is an example of CFX assembly code.
load a0, 0x7fffff
```

Many programming examples make use of definitions and macros supplied by standard header files such as `sk2_hw.inc` and `sk3_hw.cfi`. For the sake of simplicity, the `#include` statements are not shown explicitly. Note that the definitions from the header files generally use all capital letters, making it easier to differentiate them from the assembly language instructions.

This manual also includes source file examples for use with the HEAR and the HEAR Configuration Tool. These examples are shown in a bold, fixed-width font which matches the format used for CFX assembly language examples.

1.4 MANUAL ORGANIZATION

Introduction to BelaSigna 300 Programming contains the following chapters and appendix:

- **Chapter 1: Introduction**, describes the purpose of this manual, outlines how the manual is organized, and gives a list of suggested manuals to read for more information.
- **Chapter 2: Architecture Overview**, provides a basic architectural overview of BelaSigna 300.
- **Chapter 3: Design Information**, provides general information to remember when programming for BelaSigna 300.
- **Chapter 4: Introduction to Sample Code**, describes how to access and use the sample code provided with BelaSigna 300.
- **Chapter 5: Porting Guide**, describes how to port BelaSigna 250 and BelaSigna 200 based applications to BelaSigna 300. This includes using the programming tools and examples of how to program common tasks for both the CFX and RCore DSPs.
- **Appendix A: Glossary**, provides definitions for new terms and abbreviations that are introduced in this manual.

1.5 FURTHER READING

For more information, refer to the following documents:

- *Integrated Development Environment User's Guide*
- *CFX DSP Architecture Manual*
- *RCore DSP Architecture Manual*
- *Hardware Reference Manual for BelaSigna 300*
- *Firmware Reference Manual*
- *HEAR Configurable Accelerator Reference Manual*
- *CFX Toolchain Reference*

2 Architecture Overview

2.1 INTRODUCTION

BelaSigna 300 is a complete system for developing advanced portable audio devices. It provides an audio signal processing platform with numerous programmable and configurable components. It also includes firmware which can be used to help develop a complete signal processing application.

The BelaSigna 300 system is an asymmetric dual-core architecture, mixed-signal system-on-chip designed specifically for audio processing. It centers around two processing cores: the CFX Digital Signal Processor (DSP) and the HEAR Configurable Accelerator. The CFX DSP Core is used to configure the system and coordinate the flow of signal data progressing through the system. The CFX DSP may also be used for custom signal processing applications that cannot be handled by the HEAR Core. The HEAR Core is a microcode-configurable signal processing engine that works with the CFX DSP Core to execute a variety of signal processing functions. The HEAR is designed to perform both common signal processing operations and complex standard filterbanks such as the WOLA filterbank, reducing the loading on the CFX DSP Core.

The CFX and HEAR cores have a number of local and shared memories available to them. The two cores process data brought into system memory by the Input/Output Controller (IOC). The IOC can handle inputs from either an analog input stage or pulse code modulation (PCM) interface input, and can handle outputs to a digital direct-drive output stage or PCM interface output. The FIFO controller simplifies access to data by the CFX DSP, the HEAR and the IOC by providing configurable hardware-based first-in, first-out (FIFO) buffers.

A top-level block overview of the components of the BelaSigna 300 system is provided in Figure 1.

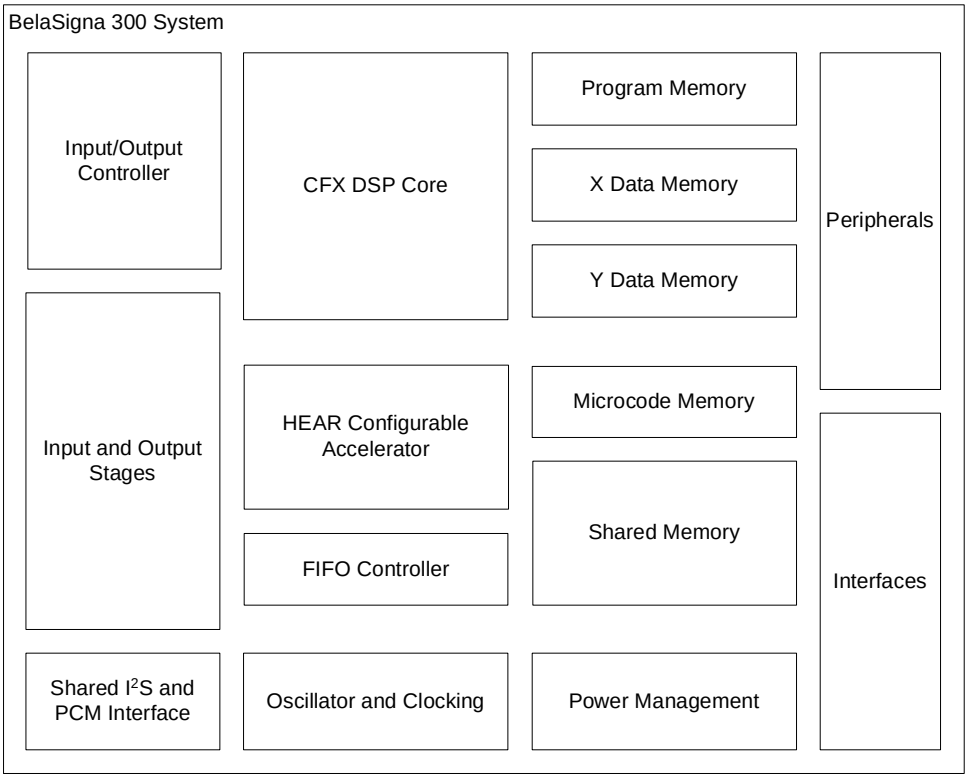


Figure 1: Top-Level System Block Diagram

More information on the various hardware components is provided in the subsections below. A brief introduction to the firmware components of the BelaSigna 300 system is also included in Section 2.13, “Firmware Components”.

2.2 CFX DIGITAL SIGNAL PROCESSOR

The CFX DSP is the primary digital processor in the system. It is a user-programmable, general-purpose DSP. The CFX serves many roles in the system: it acts as the primary controller in the system; it interacts closely with all peripherals and external interfaces; and it is used to coordinate and assist in all data and control processing tasks. The CFX DSP is closely tied to its local memories, but also has access to the HEAR Accelerator’s memories.

More information on the CFX DSP is available in the *CFX DSP Architecture Manual* and the *Hardware Reference Manual for BelaSigna 300*.

2.2.1 CFX Memory

The CFX DSP core uses a dual-Harvard memory bus architecture to provide separate buses for the program memory and two data memories.

The program memory space available to the CFX includes a program ROM as well as program RAM space to store a user application and interrupt vectors. This memory is accessible as instructions

to the CFX through the instruction memory bus and as data for the CFX through the P memory bus. Additionally the P memory space also provides access for the CFX to the HEAR core's microcode memory, as well as the system control and configuration registers.

The data memory areas available to the CFX are divided into two memory spaces that may be accessed either separately through the X memory and Y memory buses as 24-bit data, or concatenated together as 48-bit XY memory data. The X memory bus also provides access for the CFX DSP to the HEAR shared memory and configuration registers.

More information on the memories and associated memory buses is available in the *Hardware Reference Manual for BelaSigna 300*.

2.3 HEAR CONFIGURABLE ACCELERATOR

The HEAR Configurable Accelerator is a 24-bit signal processing engine that provides specialized and standard signal processing functions. It provides essential signal processing functionality such as FIR/IIR filtering, vector-based arithmetic operations, as well as updated versions of the functions available from the WOLA filterbank coprocessor found on BelaSigna 250 and BelaSigna 200. It is highly configurable at the hardware and microcode levels. The microcode for the HEAR is organized into modules which perform a single data processing task through one or more microcode functions. You can organize these functions into groups called function chains which are then triggered by hardware events such as a command from the CFX, or an interrupt from the FIFO controller. The CFX can start, pause, or reset the HEAR as needed.

The HEAR has access to shared memory for storage of inputs, outputs, coefficients and intermediate results. The HEAR also has an integrated Block Floating Point (BFP) unit, which is used when accessing the shared memory. The BFP unit automatically detects the magnitude of the data being written, and rescales the data during the next read to avoid overflow. The HEAR has dedicated microcode memory for storing the application's microcode configuration.

The execution of function chains on the HEAR is managed by the function chain controller. The HEAR Configurable Accelerator supports 24 different function chains—16 can be triggered by commands from the CFX and 8 can be triggered by interrupts from the FIFO controller (one per FIFO). The CFX can configure the function chain controller to control which function chains are active as well as the priority of the different function chains. The CFX can receive up to eight interrupt signals from the HEAR to indicate the completion of individual microcode functions or entire function chains. As part of the microcode configuration process, you can configure the specific events to trigger the function chains, as well as the timing of the interrupt signals that the HEAR sends back to the CFX.

For more information on the HEAR Configurable Accelerator and its associated hardware, consult the *Hardware Reference Manual for BelaSigna 300*. For more information on microcode configuration and available functions, consult the *HEAR Configurable Accelerator Reference Manual*.

2.3.1 Shared Memory

The shared memory provides data storage for the HEAR Configurable Accelerator. It is also accessible from the CFX and acts as the primary means of interfacing between the two cores. All HEAR shared memory is 48-bits wide and the CFX can address individual 24-bit words within each memory. All memories are single-ported and the shared memory bus controller controls access to them.

Shared memory is composed of the following distinct physical memories:

- H0, H1, H2, H3, H4, and H5 memory are all 128x48-bit memories generally used for filterbank processing such as analysis results, gain values and scratch memory.
- A and B memory are 1024x48-bit memories which are connected to the FIFO controller, and hence are generally used for FIFOs.
- C and D memory are 1024x48-bit memories generally used for coefficients as well as microcode configuration data and microcode module state variables.

The memories described above can also be accessed in an interleaved fashion. This interleaving is done at the 48-bit word level (i.e., alternating 48-bit words come from two different physical memories). This allows the HEAR to access data 96-bits at a time and in a manner that is advantageous for certain algorithms, such as accessing samples and coefficients in an interleaved fashion.

For more information on the shared memory, consult the *Hardware Reference Manual for BelaSigna 300*.

2.4 FIFO CONTROLLER

The FIFO controller controls access to the A and B shared memories, allowing portions of this memory to act as hardware managed FIFO buffers. This controller serves three purposes in managing access to shared memory:

- It remaps portions of the address space so that a hardware circular buffer is implemented, and the most recent samples in the buffer can always be accessed at the same logical address.
- It resolves access conflicts between the input/output controller (IOC) and the two system cores that access these memory areas through the shared memory bus.
- It tracks incoming data to a buffer or outgoing data from a buffer. Changes to these buffers can trigger function chains on the HEAR or interrupts on the CFX.

More information on configuring FIFOs and using the FIFO controller is available the *Hardware Reference Manual for BelaSigna 300*.

2.5 INPUT/OUTPUT CONTROLLER

The Input/Output Controller (IOC) manages the transfer of audio data into and out of the system through the input stage, output stage and PCM interface. The IOC can be configured to acquire input data from any combination of the four analog input channels belonging to the input stage and the two PCM input channels. The IOC is also used to direct output data to any combination of the direct digital output channel and the two output PCM channels. The IOC routes the data to and from the HEAR shared memory via the FIFO controller. This allows you to organize the IOC input and output data into various combinations of single-channel and multi-channel FIFO buffers.

More information on the configuration and operation of the IOC is available in the *Hardware Reference Manual for BelaSigna 300*.

2.6 INPUT STAGE

The BelaSigna 300 system's input stage provides four configurable audio input channels. Each input channel has an input multiplexer, a programmable pre-amplifier, a low-pass anti-aliasing filter, an oversampled A/D converter and a high-quality digital decimation filter. Configurable line-outs are also available for monitoring the pre-amplified and low-pass filtered input signals. The input channels provide 18-bit digital samples to the rest of the BelaSigna 300 system via the IOC. The input stage AD converters allow for configurable sampling rates and configurable sampling delays between channels.

More information on the configuration and usage of the input stage is available in the *Hardware Reference Manual for BelaSigna 300*.

2.7 OUTPUT STAGE

The BelaSigna 300 system's output stage is composed of a single configurable audio output channel. This output channel uses a high-quality interpolation filter, an ultra-low power sigma-delta modulator and a direct digital output to drive an external speaker or receiver. The output stage is provided with 18-bit digital samples from the rest of the BelaSigna 300 system via the IOC. The direct digital output driver can be configured in a high-power output mode for increased output power.

More information on the configuration and usage of the output stage is available in the *Hardware Reference Manual for BelaSigna 300*.

2.8 PCM INTERFACE

The BelaSigna 300 system has a configurable pulse code modulation (PCM) interface which can also operate in an Inter-IC Sound (I^2S) compatible mode. The PCM interface is highly configurable, allowing it to interface with a wide variety of external devices.

This interface provides two additional digital inputs and two additional digital outputs for audio data. The IOC treats these inputs and outputs in a similar manner to the input and output stage signals. The PCM interface can also be used directly by the CFX DSP for a finer degree of control in the interface communications.

More information on the configuration and usage of the PCM interface is available in the *Hardware Reference Manual for BelaSigna 300*.

2.9 PERIPHERALS

The BelaSigna 300 system provides a number of peripherals that are controlled through the CFX DSP core. The system peripherals include:

Interrupt Controller

The interrupt controller manages the signalling of interrupts from external interfaces and peripherals to the CFX DSP. Interrupt vectors are stored in P memory at a fixed location.

General-Purpose Timers

Two general-purpose timers are available in the BelaSigna 300 system. Each timer can be configured for a range of frequencies and granularities. Each timer can also operate in a free-run or one-shot mode.

Watchdog Timer

The watchdog timer is used to monitor the application's execution. Two successive watchdog timeouts triggers an automatic reset of the system.

Cyclic Redundancy Code (CRC) Generator

A 16-bit CRC generator is included to aid with data integrity testing. The system firmware extensively uses the CRC generator but it can also be useful in end-user applications.

Access Mode Security Blocks

The BelaSigna 300 system includes a restricted access mode which can be used to secure intellectual property in the form of algorithms or data stored in memory. Data in non-volatile memory can also be stored in an encrypted format and decoded during system startup.

More information on configuring and using the peripherals is available in the *Hardware Reference Manual for BelaSigna 300*.

2.10 EXTERNAL INTERFACES

The BelaSigna 300 system provides a number of external interfaces which can be controlled through the CFX DSP. These interfaces provide support for different types of external communications, as well as signalling through general-purpose input/output (GPIO) pads and low-speed analog to digital (LSAD) converters. The external interfaces provided include:

Inter-IC Communications (I^2C) Interface

An I^2C interface is provided for debugging and general-purpose communications. A single I^2C bus is shared between the debug I^2C interface and the general-purpose I^2C interface. The debug I^2C interface is the hardware-based debugging interface that is accessed through a fixed I^2C address and is tied directly to the CFX DSP and its memory components. A general-purpose I^2C interface is available for using the BelaSigna 300 system as an I^2C master or slave. This interface can operate at speeds up to 400 kbps.

General-Purpose Input/Output (GPIO) Interface

Five software controlled GPIO pins are available on the BelaSigna 300 system. Transitions on GPIO pins can be used to trigger interrupt service routines (ISRs) on the CFX DSP.

Low-speed Analog-to-Digital (LSAD) Interface

The LSAD interface provides a low-frequency, 10-bit interface for sampling slow-varying signals. The LSAD interface can be connected to five external signals and three internal voltage signals for tasks such as battery monitoring.

Serial Peripheral Interface (SPI)

The SPI external interface is used to communicate with other SPI-compatible external devices. In particular, this interface is used to communicate with external non-volatile memory devices such as an EEPROM. The system firmware, which provides EEPROM access, uses this interface extensively.

Universal Asynchronous Receiver-Transmitter (UART) Interface

A general-purpose two-pin UART interface is available for RS-232 compatible communications. The baud rate (bits/second) of this interface is typically configurable within a range of 0.4 to 320 kbps, depending on the application's system clock.

Note that some physical pins used for these interfaces are multiplexed, so not all interface capabilities are available simultaneously in an application. More information on configuring and using the external interfaces is available in the *Hardware Reference Manual for BelaSigna 300*.

2.11 CLOCKING

The BelaSigna 300 system is normally clocked by an external clock source. The system supports a variety of clock frequencies up to 40 MHz, at a 1.8 V supply voltage. It also includes a 17 kHz standby oscillator for an ultra-low power standby mode.

A number of clock domains are derived from the external clock source to operate the input stage, output stage, peripherals and external interfaces. More information on these clock domains and their configuration is available in the *Hardware Reference Manual for BelaSigna 300*.

2.12 POWER MANAGEMENT

The BelaSigna 300 system is powered from a single battery voltage and generates a number of regulated supply voltages for different system components. These regulated voltages provide low-noise power supplies to the analog components of the system and external devices such as a microphone. The system also includes integrated power management and battery monitor components to ensure proper system operation under varying power supply conditions.

More information on power supply issues and the power management components is available in the *Hardware Reference Manual for BelaSigna 300*.

2.13 FIRMWARE COMPONENTS

The BelaSigna 300 firmware is supplied as a set of include files and libraries which are accessible using the ON Semiconductor SignaKlara® IDE. The libraries are included as source code, compiled object code and in some cases as ROM-based routines. The firmware is used to help build applications for the BelaSigna 300 device. It is composed of the following major components:

System Include Files and Macros

The system include files provide preprocessor definitions for all major hardware components. These definitions provide labels and names for things like addresses of digital and analog registers, locations and sizes of bitfields and standard settings for bitfields. These can be used when configuring and accessing the various hardware components from a CFX

DSP program. The system include files also provide preprocessor macros which further simplify the use of each hardware component. Using the standard system include files and macros increases the readability of your application's code and can simplify porting applications to future versions of BelaSigna 300.

System Library

The system library provides several useful functions that can be called from your application. The system library is used for tasks such as resetting the system or changing the operating clock frequency.

Math Library

The math library provides a set of optimized math functions which are useful in developing audio signal processing applications. These functions include basic mathematical operations such a log(x), e^x, square root, sine, cosine, reciprocal and division. The available functions also include signal processing operations such as attack/release filtering and various conversions between root-mean-square, power and decibel values. Some functions work with both scalar and vector inputs. These functions are helpful when developing the audio signal processing portion of your application.

EEPROM Library

The EEPROM library provides a set of standard functions for communicating with an EEPROM connected to the BelaSigna 300 device's SPI port. This library provides high-level functions for configuring the EEPROM and for reading or writing data in different word and block sizes. The EEPROM library is useful for developing portions of an application that require access to non-volatile memory storage.

File System

A standard EEPROM-based file system structure is used for BelaSigna 300 applications. The file system structure is supported by the Program ROM and the standard Bootloader firmware. It provides a structured framework for storing system data, application code and data tables. Portions of the file system can be encrypted using the AES encryption algorithm to provide protection of intellectual property in the form of application code or data.

Program ROM

The Program ROM is responsible for the booting and initialization of the BelaSigna 300 system. It also includes ROM-based versions of many of the other firmware libraries. The use of ROM-based libraries helps to reduce the program memory and EEPROM space required for your application.

Bootloader

A standard bootloader is provided with the BelaSigna 300 firmware. This bootloader provides support for loading and starting a user application stored in EEPROM using the BelaSigna 300 file system.

More information on the BelaSigna 300 firmware is available in the *Firmware Reference Manual*.

3 Design Information

3.1 Introduction

For all new users of BelaSigna 300, the following sections provide design recommendations from a programmer's perspective for making efficient use of the chip's resources. If you have used BelaSigna 250 or BelaSigna 200 before, you will also be interested in how the efficient design and implementation of signal processing algorithms on the RCore and WOLA filterbank coprocessor (referred to as the WOLA) maps to the new architecture which uses a CFX and HEAR Accelerator.

3.2 Then and Now

BelaSigna 300 is about power-efficient design, miniscule power expenditure, and clever algorithm design partitioning. Like BelaSigna 250 and BelaSigna 200, BelaSigna 300 uses a dual processor approach to provide power-efficient computation.

BelaSigna 250 and BelaSigna 200 have a general-purpose DSP core, as well as the WOLA filterbank coprocessor that performs filtering efficiently. The WOLA is key to low power consumption because its frequently performed functions are coded in hardware and designed for efficiency. Using the WOLA in conjunction with the RCore provides both processing power and flexibility.

Compared to the RCore, the CFX is a more powerful general-purpose programmable DSP/ micro-controller core. The HEAR Configurable Accelerator fulfills the function of the WOLA, but has even more capabilities as described in the *HEAR Configurable Accelerator Reference Manual*.

The HEAR provides the capability for better load-balancing because it is more flexible than the WOLA filterbank, easier to configure, has enhanced time domain processing capabilities, and flexible, user-controlled data flow.

See Chapter 2, "Architecture Overview" on page 3 for an overview of BelaSigna 300 architecture.

3.3 Programming a Dual-Core System

Optimum power efficiency is achieved by using both cores equally, or load-balancing. The amount of power consumed is primarily determined by the core that does most of the work, so if that work is evenly divided between the two cores, you can reduce the overall clock frequency, consuming less power, as illustrated in Figure 2. Since the HEAR has extensive capabilities, you can take apart your algorithm and assign different portions of it to run on the CFX or the HEAR (called *algorithm partitioning*). The vectorized, number-crunching portions should run on the HEAR, and everything else should run on the CFX. Therefore, algorithm partitioning is key to optimizing the power efficiency of programs running on BelaSigna 300.

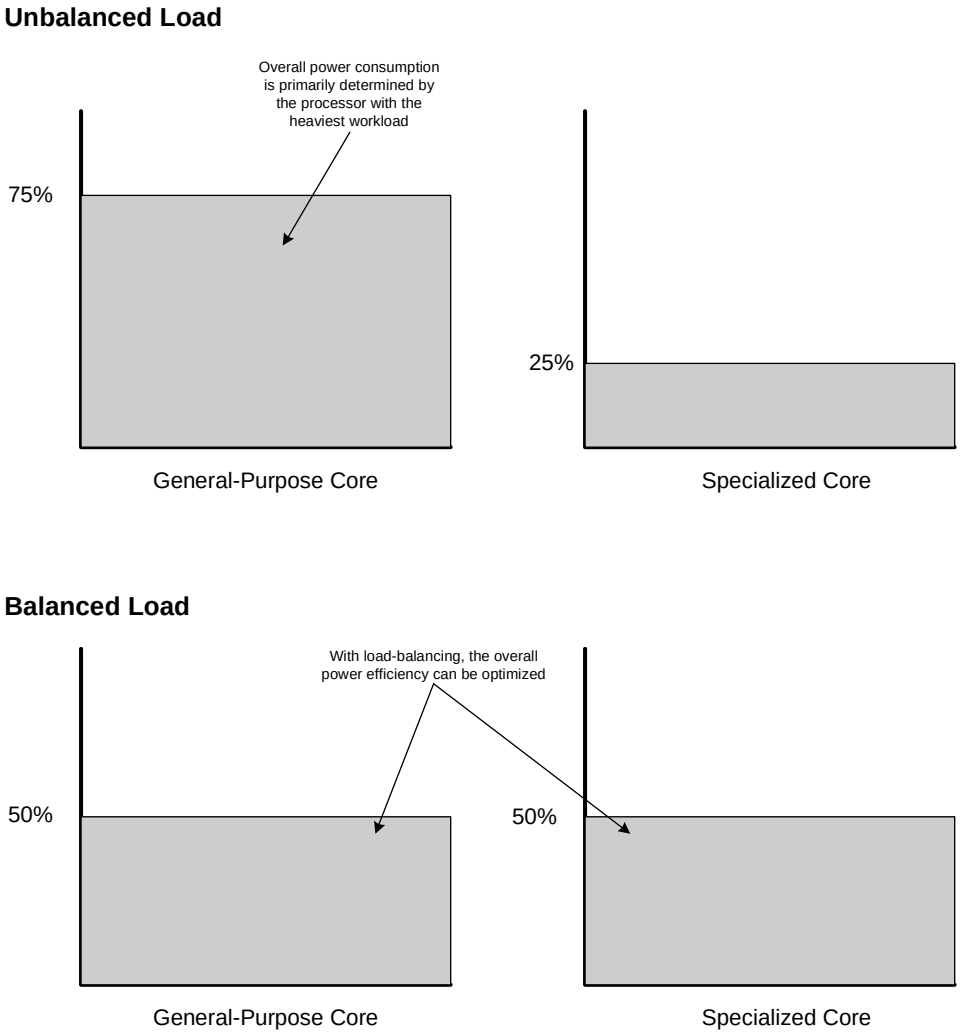


Figure 2: A Conceptual Illustration of Load-Balancing

3.3.1 Partitioning Algorithms

Partitioning algorithms for BelaSigna 300 is somewhat related to task scheduling for multiprocessor systems, a topic that has been studied extensively in the field of multiprocessor computing. This section describes a few key concepts that are useful to developers working on partitioning algorithms for BelaSigna 300.

Partitioning algorithms for BelaSigna 300 can be typically thought of as scheduling tasks among the CFX and the HEAR so that the workload is distributed among the two processors as evenly as possible. When analyzing an algorithm for partitioning, it is important to consider the algorithm from two perspectives: functional partitions and timing partitions.

Functional partitions Divide the algorithm based on the type of operations (or "functions") that need to be run

Timing partitions Divide the algorithm based on when a part of the algorithm needs to be run, and how often and how quickly it needs to be completed.

When analyzing your algorithm from the functional perspective, keep in mind that:

- The HEAR is typically more power efficient than the CFX in running the functions provided by the microcode modules. If the cycle count for a given task is about the same on both cores, the HEAR is still likely to be more power efficient. Thus it is usually better to partition your algorithm in such a way that you can utilize the functions available from the HEAR and not duplicate the same functions on the CFX. See the *HEAR Configurable Accelerator Reference Manual* for the complete description of the available microcode modules.
- There are 8 interrupts on the CFX associated with the HEAR. You can configure each interrupt to be raised by the HEAR at various points within a function chain, supporting an elaborate communication scheme between the CFX and the HEAR. For example, a function chain could contain an FIR filter function followed by a filterbank analysis function. It is possible to configure the HEAR to raise an interrupt on the CFX after the FIR filter function has completed, and another interrupt after the filterbank analysis function has completed.
- As the HEAR is designed to specialize in certain arithmetic and data processing operations, it cannot access any peripherals or external interfaces. Thus, the CFX must handle initialization and control of all peripherals and external interfaces on BelaSigna 300.

Furthermore, when analyzing your algorithm from the timing perspective, keep in mind that:

- A new block of audio data must be available at the block rate to maintain the continuity of the audio output. Thus, the critical path of your task schedule is typically constrained by the block rate. The block rate is calculated as the sampling frequency divided by the input block size. For example, a 16 kHz sampling frequency divided by a block size of 8 samples equals 2 kHz, meaning that a new block of audio data must be available every 0.5 ms.
- *Time-slicing* is a technique for partitioning your algorithm over the time dimension, to free up clock cycles for other more critical computations. It involves spreading parts of your algorithms over several blocks instead of having all the computations done within one block. For example, in a dynamic range compression algorithm, instead of calculating new compression gains for all 16 frequency bands within one block, time-slicing divides up the algorithm so that only the compression gains for bands 1-4 are calculated in the first block, then only the gains for bands 5-8 are calculated in the next block, and so forth. This way, the algorithm is partitioned so that it only needs to calculate the compression gains for 4 frequency bands at a time. Implementing time-slicing might compromise the performance of your algorithm if it is not implemented in a way that accounts for the differing sample rates needed.
- Either a command from the CFX or an event from the FIFO controller can launch a function chain on the HEAR. Thus it is important to consider the flow of the data in your algorithm to see if the FIFO controller should directly launch a function chain on the HEAR, while the CFX is running other computations.

3.4 DATA FLOW

With BelaSigna 250 and BelaSigna 200, the data flow is optimized for algorithms that employ data going from the input to the output: the Input/Output Processor (IOP) controls the flow of data that comes from the input stage, to the input FIFO, through processing, to the output FIFO, and finally the output stage. However, many signal processing algorithms have been developed, such as adaptive feedback cancellation, which require picking up data from the output FIFO, processing the data, and then subtracting it from or somehow combining it with the data in the input FIFO. Moreover, some other algorithms might require data that arrives from the PCM interface, and

need to process and/or combine it with other data in the input or output FIFO. With BelaSigna 250 and BelaSigna 200, managing the data flow in these kinds of algorithms can require significant computation and memory overhead. Such algorithms demand a more flexible data flow.

BelaSigna 300 features significantly more flexible data flow through the Input/Output Controller (IOC) and the FIFO controller. The IOC can be configured to automatically route data between the system memory on one side, and the input/output stage and PCM interface on the other side. The FIFO controller supports up to 8 FIFOs that can be configured to different sizes and locations within the system memory. Each of the FIFOs can be connected to one or more data sources or sinks. A data source is simply something that writes data into the FIFO, and a data sink reads data from the FIFO. For example, the input stage is a data source for an input FIFO, whereas a PCM port can both receive and transmit data, so it can be both a data source and a data sink. Moreover, a data source or sink is not required to be a physical interface connected to the IOC. An FIR filter, for example, running on the HEAR can be a data sink that reads from an input FIFO, while at the same time it can be a data source for another FIFO. FIFOs that are connected to a physical interface through the IOC are called *hardware FIFOs*. FIFOs that are controlled by the software on the CFX and HEAR are called *software FIFOs*. The IOC, CFX and HEAR all have access to the FIFOs via the FIFO controller, which maintains a fixed access priority scheme. See the Hardware Reference Manual for details about the IOC and FIFO controller, and also Section 5.4, “FIFO Controller and IOC Configuration” on page 51.

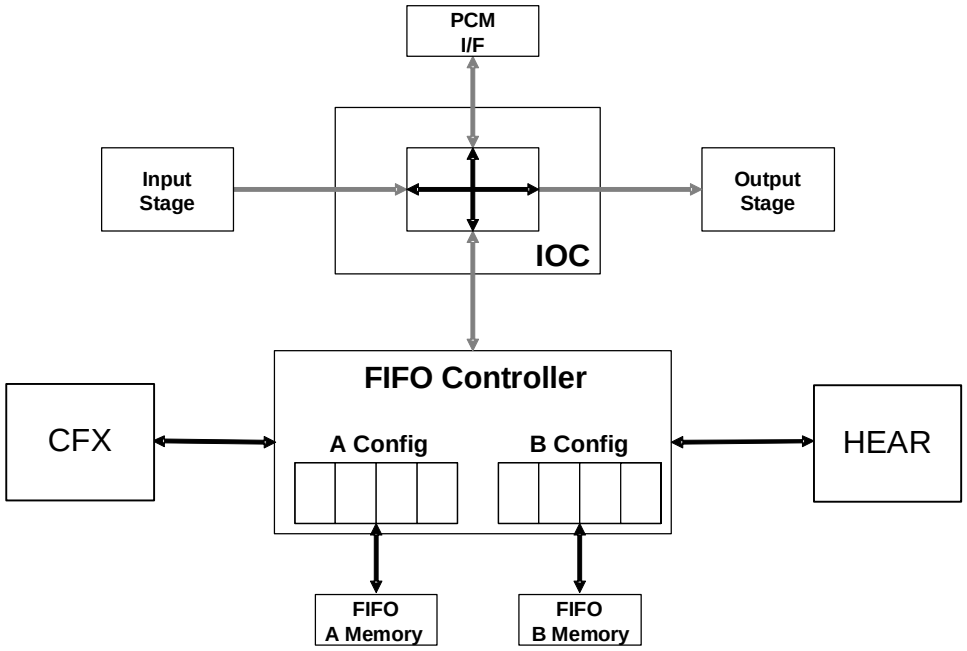


Figure 3: Flexible Data Flow via the IOC and FIFO Controller

To design your algorithms to take advantage of the flexibility provided by the IOC and FIFO controller, consider all the components that can be data sources, data sinks, or both. Then, configure the IOC and the FIFO controller such that they connect the data sources and sinks to the FIFOs that you have specified. Access to the FIFOs from within your program is straight-forward—the FIFO controller handles all the pointer updates to the FIFOs, and automatically maps direct FIFO access to the appropriate location in the physical memory, so that your program does not need to calculate new addresses due to data updates in the FIFOs. For example, an input FIFO can

be configured in such a way that the most recent block of data from the input stage is always located at the base of the FIFO. Thus, an FIR filter that processes data from the input FIFO can simply start from the same memory address every time a new block of data is available in the FIFO.

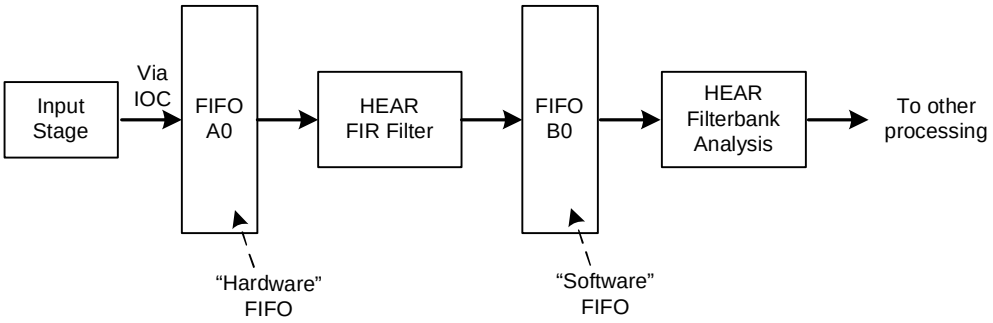


Figure 4: Hardware FIFO vs Software FIFO

This page left blank for printing purposes.

4 Introduction to Sample Code

4.1 Introduction

To help you learn about the various programming aspects of BelaSigna 300, the Integrated Development Environment (IDE) includes a number of sample code applications as part of its installation. Each sample code application is intended to be simple, easy to use and modify, while at the same time, is also practical and applicable. You might want to experiment with programming BelaSigna 300 by building an application based upon a sample code template. Nevertheless, while the sample code does contain recommended programming practices for BelaSigna 300, it is primarily intended to be a learning tool and does not represent the most efficient or optimal way to implement programs for BelaSigna 300.

Note: Additional sample code is available in the Communication Toolkit (CTK) Developer Kit. It includes sample code applications for communicating with a host system such as a PC.

4.2 Accessing the Sample Code

The sample code applications are included in the IDE installation. To view the source code outside of the IDE, you can find it in the directories listed in Table 1.

Alternatively, the sample code can be viewed through the IDE. To modify, build and run a particular sample, follow the steps in Method #1.

If you want to import all of the sample applications into the IDE at once, and do not need the ability to build the applications, follow the steps in Method #2.

A text file called `readme.txt` accompanies each sample code application. This file contains information about how that specific sample works.

Note: The default installation folder is specified in the *Integrated Development Environment User's Guide*. Your installation folder is represented as **<EDK_HOME>** in this document.

Table 1: Installed Location of Sample Applications

Sample Application	Location of Compiled Application and Source Code
Battery Monitor	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\battery_monitor
Data Logging	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\data_logging
Input Dynamic Range Extension	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\idrx
Mono FIFO Copy	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\mono_fifo_copy
Mono Filterbank Passthrough	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\mono_filterbank

TABLE 1: INSTALLED LOCATION OF SAMPLE APPLICATIONS (CONTINUED)

Sample Application	Location of Compiled Application and Source Code
Multi-Filterbank Passthrough (Mixed)	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\multi_filterbanks_mixed
Multi-Filterbank Passthrough (Mono)	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\multi_filterbanks_mono
PCM Passthrough	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\pcm_passthrough
Sampling Delay	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\sampling_delay
Simple-Stereo Filterbank Passthrough	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\simple_stereo_filterbank
Single-Channel WDRC	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\single_channel_wdrc
Soft Power Down	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\soft_power_down
Stereo External DAC Output	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\ stereo_external_DAC_output
Subband FIR/IIR Filtering on the CFX	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\subband_filter_cfx
Subband FIR/IIR Filtering on the HEAR	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\subband_filter_hear
Time-Domain FIR/IIR Filtering on the CFX	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\td_filter_cfx
Time-Domain FIR/IIR Filtering on the HEAR	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\td_filter_hear
Tone Generation	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\tone_generation
Two-Channel WDRC	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\two_channel_wdrc
Volume Control Application	**<EDK_HOME>**\source\sk3\samples\BelaSigna300\volume

Method #1: Creating a copy of a single sample code application in the workspace

1. From the menu, select **File > New > Project...**.
2. Select **SignaKlara Development / SignaKlara Development Project** and click **Next**.
3. Type in a project name and click **Next**.
4. Select **BelaSigna 300 Projects** from the drop down list.
5. Expand **Sample Code Projects** and select the sample code template that you wish to view. You can read about the sample on the right side of the dialog.
6. Click **Finish**. The IDE creates a project in your workspace using the sample code template.

This method copies one sample code application at a time into your workspace. You can use this copy as a template for your own applications.

Method #2: Viewing multiple sample code applications through the IDE (no copies)

1. From the menu, select **File > New > Project**.
2. Select **General > Project** and click **Next**.
3. Type in a project name. You might want the project name to indicate that this project will contain multiple sample code projects.

4. Uncheck the **Use default location** check box and browse to the location of the sample code applications in your file system. The location is **<EDK_HOME>**\source\sk3\samples\BelaSigna300. Then click **Next**.
5. Under **Referenced projects**, leave all projects unchecked. Click **Finish**.

These projects have not been copied into your workspace; they are the original versions from the IDE installation. As such, they are read only. Within each sample code application there is a master directory that contains the executable for this sample code application. You can run and step through the sample code application on the BelaSigna 300 hardware if you wish.

4.3 ACCESSING THE FIRMWARE LIBRARIES

The sample code applications use a number of macros from the firmware libraries, particularly in the initialization routine. Table 2 specifies where documentation and source code for these libraries can be found.

TABLE 2: MORE INFORMATION ABOUT FIRMWARE LIBRARIES

Macro Type	Documentation Source	Source Code Location
General Firmware Macros	*Firmware Reference Manual*	**<EDK_HOME>**\include\sk3
Math Library Macros	*Firmware Reference Manual - Chapter 5*	**<EDK_HOME>**\include\sk3\sk3_math.cfi **<EDK_HOME>**\source\sk3\mathlib

4.4 CODE STRUCTURE

The BelaSigna 300 system is very flexible, and thus, there are many different ways in which an application might be structured. However, some application structures are more prone to errors than others. The program structure used in the sample code applications has been selected because it is robust and easy to modify.

The HEAR microcode is divided into function chains. Each function chain is associated with a particular command from the CFX or an event from the FIFO controller. The processing on the CFX is divided into a number of core blocks that occur relative to these function chains on the HEAR. The CFX transitions from one block or state to the next based on interrupts it receives from the HEAR or FIFO controller.

The structure used by all sample code applications is shown in Figure 5 below.

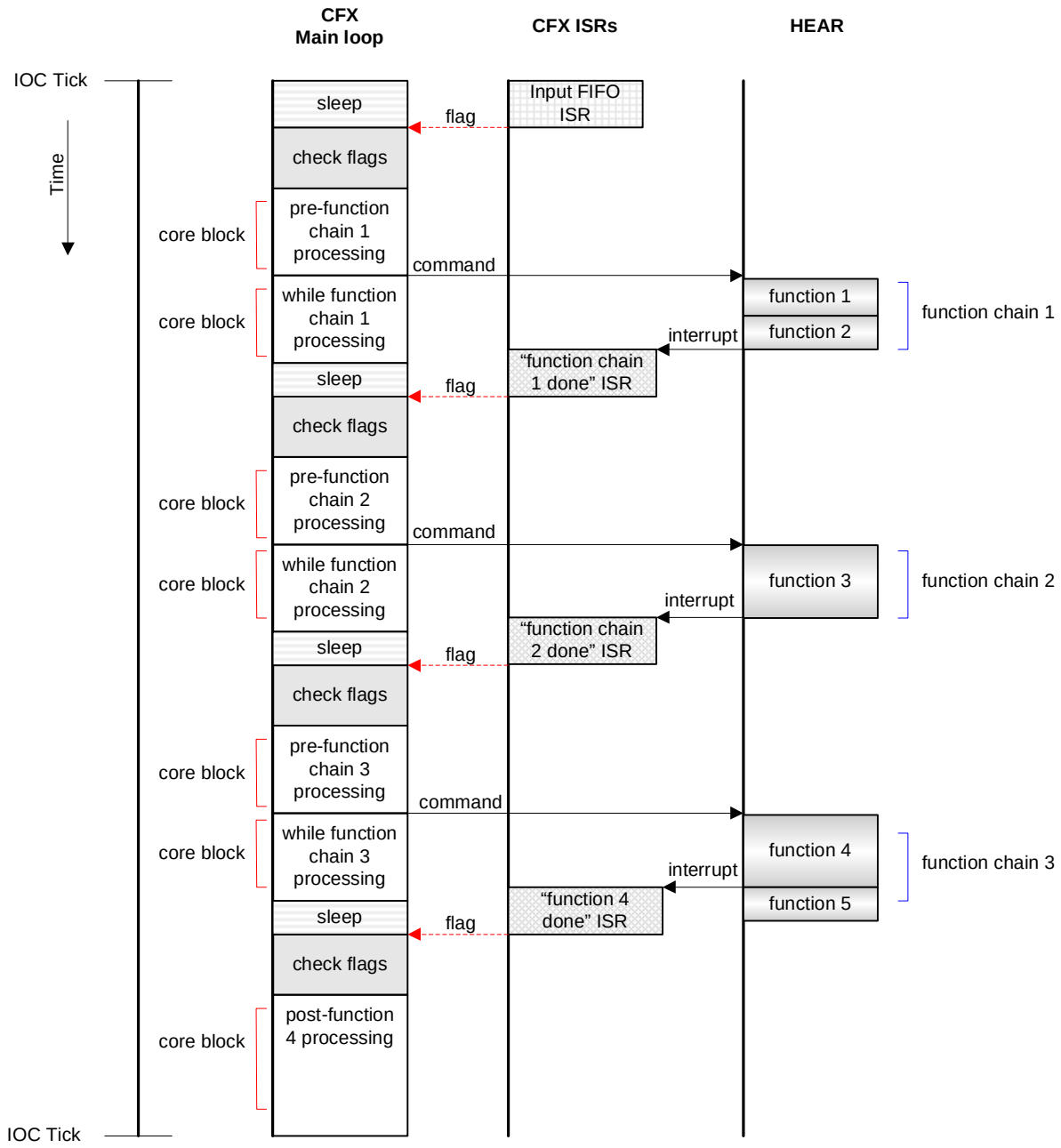


Figure 5: Sample Code Structure

Figure 5 illustrates how the processing on the CFX can be grouped into core blocks. These blocks occur relative to the function chains on the HEAR. The example above has three function chains. An interrupt is sent by the HEAR to the CFX at the end of function chains 1 and 2 and part-way through function chain 3 (at the end of function 4). Thus, the processing on the core can be divided into 7 distinct blocks:

- pre-function chain 1 processing
- while function chain 1 processing
- pre-function chain 2 processing
- while function chain 2 processing
- pre-function chain 3 processing
- while function chain 3 processing
- post-function 4 processing

These blocks are found in the `app.cfs` source file.

In each sample code application, the CFX launches the function chains from the main loop. When an interrupt is received by the CFX, a flag is set in the appropriate interrupt service routine (ISR). These ISRs are found in the `app_isr.cfs` source file. There is a separate flag for each interrupt.

The main loop of each sample code application is structured as a very simple state machine. At the top of the loop, the program checks whether any interrupts have been received by checking the interrupt flags. The flags are checked in the same order as the HEAR and FIFO interrupts are expected to arrive. The code then branches to the appropriate core block based on which flag has been set. Once the core block has completed execution, the flags at the top of the main loop are checked again. If no additional flags have been set, the CFX goes to sleep to save power. If another flag has been set, the code branches to the appropriate core block. The code for each core block is also found in the `app.cfs` source file.

For this state machine to work, command priorities on the HEAR and interrupt priorities on the CFX must be carefully considered. On both the CFX and the HEAR, there are two priority levels: normal and high. If all events have the same priority level, lower numbered events are treated with a higher priority level. For example, on the CFX, it is important that the HEAR 0 interrupt corresponds to the first interrupt expected from the HEAR, the HEAR 1 interrupt corresponds to the second interrupt expected from the HEAR, and so on. Similarly, it is important that command 0 corresponds to the first function chain to run on the HEAR, command 1 corresponds to the second function chain to run on the HEAR, and so on. For more information on interrupt priorities on the CFX and command priorities on the HEAR, refer to the *Hardware Reference Manual for BelaSigna 300*.

4.5 FILE STRUCTURE

All sample code applications have a common source file structure. The files that are common to all sample code applications are described below.

4.5.1 CFX Source Code File Structure

The files that are common to the CFX portion of all sample code applications are described in Table 3.

TABLE 3: COMMON FILES IN SAMPLE APPLICATIONS

File Name	Description
app.cfs	This file is the main source file for the CFX software. It contains the main loop of the application and the core blocks. This file can also contain application-specific functions such as filters.
app.cfi	This file is the include file for app.cfs. It includes numerous defines for the hardware and software configuration. It also declares global variables and functions.
app_init.cfs	This file contains the source code for the application's hardware initialization function, named Initialize.
app_isr.cfs	This file contains the source code for all of the application's interrupt service routines. There will be an interrupt service routine for each interrupt received by the CFX from the HEAR, the FIFO controller, peripherals or interfaces.
app_memory.cfs	This file contains all of the memory allocation for the application.

4.5.2 HCT Input File Structure

There is a single input file to the HEAR Configuration Tool (HCT): microcode.hct. This file is used to configure the microcode that runs on the HEAR. For more information about the HCT, see Section 5.5, "HEAR Configuration" on page 53.

4.6 APPLICATION INITIALIZATION

Each sample code application has an initialization routine that can be found in the app_init.cfs source file. This initialization routine does the following:

- Configures the analog blocks
- Configures the FIFOs and the Input/Output Controller (IOC)
- Configures the HEAR's function chain controller
- Configures the CFX's Interrupt Controller
- Other system configurations
- Starts the HEAR

The initialization routine makes extensive use of the firmware macros. Refer to Section 4.3, "Accessing the Firmware Libraries" on page 19 for more information about how to access these macros. The inputs to these macros are defines which are specified in the include file app.cfi.

The initialization routine is called in the app.cfs source file at the very beginning of the program (which is indicated by the label main). Once initialization has completed, code execution continues to the waitloop, where it remains until an interrupt occurs to start the state machine.

4.7 MODIFYING THE SAMPLE CODE APPLICATIONS—AN EXAMPLE

To modify a sample code application, the application must be imported into the IDE using Method #1, as described in Section 4.2, “Accessing the Sample Code” on page 17.

One of the most challenging modifications that can be made to a sample code application is the addition of a function chain on the HEAR. This section describes the minimum steps that are involved in making this change to the structure.

Consider the following example. The sample code structure illustrated in Figure 5 on page 20 is to be modified through the addition of function chain 4. The new structure will appear as shown in Figure 6 below.

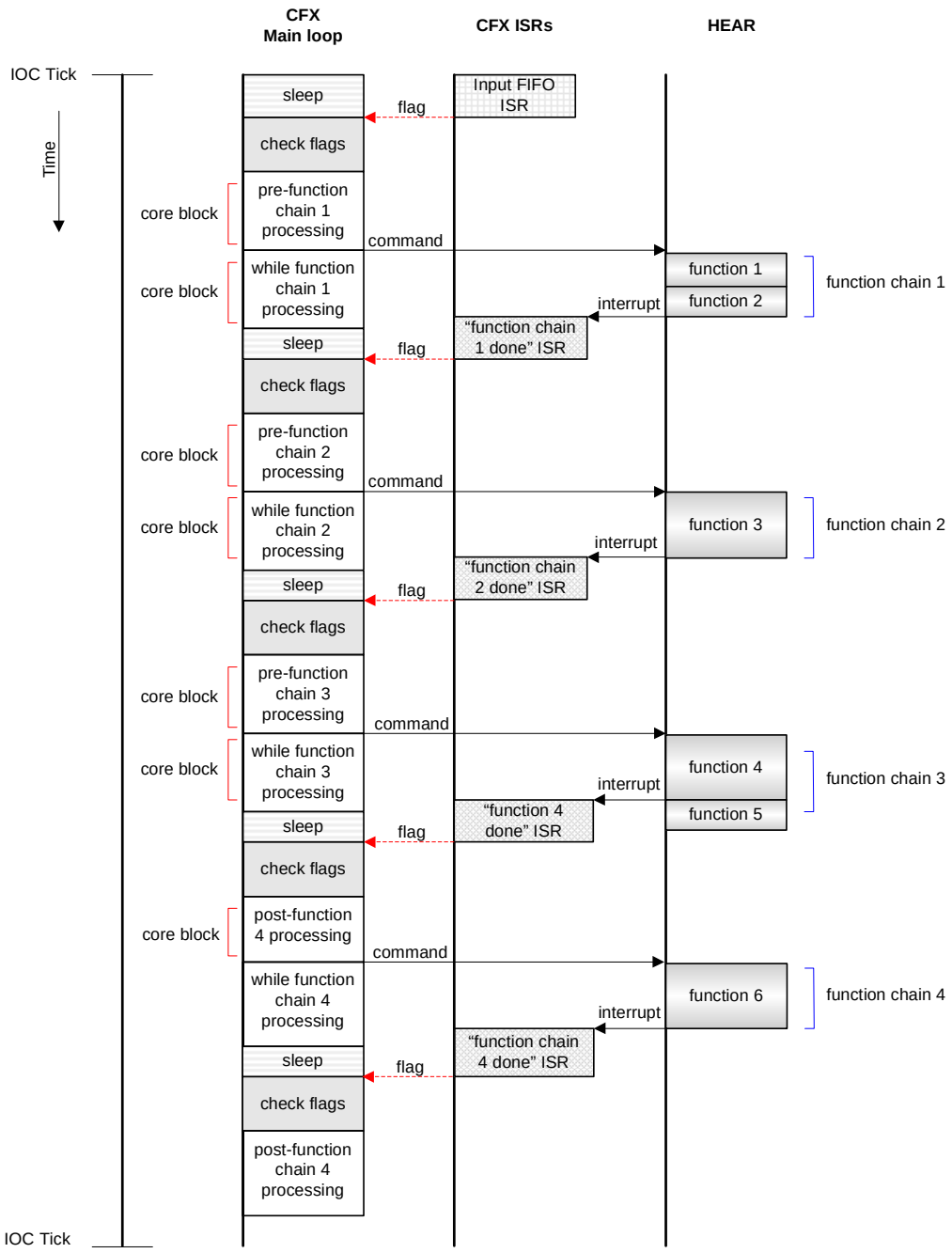


Figure 6: Modified Sample Code Structure

To add function chain 4 to the structure, follow these steps:

1. Add function chain 4 to the `microcode.hct` file:

```
//
// Function Chain 4
//
StartFunctionChain(CMD4, function_chain4)

        // Call some function (corresponding to function 6 in Figure 6)
        …

        // Trigger the HEAR 4 interrupt on the CFX
        Event(4)

EndFunctionChain()
```

In this example, command 4 from the CFX starts this function chain. The function chain triggers the HEAR 4 interrupt on the CFX upon completion.

The HCT uses this function chain definition to automatically update the define HEAR_FC_ENABLE_VAL, which can be found in microcode.cfi. This define specifies which function chains are enabled on the HEAR.

2. Add the HEAR 4 ISR to app_isr.cfs.

   Since function chain 4 triggers the HEAR 4 interrupt on the CFX, add an ISR for this interrupt. This ISR must set the appropriate flag to indicate that this interrupt has occurred. An example ISR is as follows:

```
// ------------------------------------------------------------------------
// Function:       HEAR_4_ISR
// Description:    Sets a flag indicating that function chain 4 has completed
//                 on the HEAR
// Clock cycles:   3
// Modifies:       None
// ------------------------------------------------------------------------
HEAR_4_ISR:

    returni.2d

    // Set the HEAR function chain 4 done flag
    store x[X_HEAR_FUNCTION_CHAIN4_DONE], 1     // [ds1]
    nop                                         // [ds2]
```

3. Add HEAR_4_ISR to the interrupt vector table in app_isr.cfs.

   This entry in the interrupt vector table must change from:

```
goto Empty_ISR      // HEAR 4
```

   to:

```
goto HEAR_4_ISR     // HEAR 4
```

4. Allocate memory for the new flag X_HEAR_FUNCTION_CHAIN4_DONE in app_memory.cfs.

   Add the following to X_System_Memory_Reserved segment to allocate memory for X_HEAR_FUNCTION_CHAIN4_DONE and to initialize its value to 0:

```
// Initialize X_HEAR_FUNCTION_CHAIN4_DONE to 0
Sys_Mem_Reserve_Init(X_HEAR_FUNCTION_CHAIN4_DONE, 1, 0)
```

5. Specify X_HEAR_FUNCTION_CHAIN4_DONE as an externally defined symbol in app.cfi.

Add the following line to `app.cfi` so that the flag `X_HEAR_FUNCTION_CHAIN4_DONE` can be referenced in all the files that include `app.cfi`:

```
.declare X_HEAR_FUNCTION_CHAIN4_DONE
```

6. Modify the `INT_ENABLE_VAL` define in `app.cfi` to enable the HEAR 4 interrupt on the CFX.

   Change the define `INT_ENABLE_VAL` from this:

```
#define INT_ENABLE_VAL       (INT_EBL_MASTER   | \
                              INT_EBL_WATCHDOG | \
                              INT_EBL_HEAR_0   | \
                              INT_EBL_HEAR_1   | \
                              INT_EBL_HEAR_2   | \
                              INT_EBL_HEAR_3   | \
                              INT_EBL_FIFO_A0)
```

   to this:

```
#define INT_ENABLE_VAL       (INT_EBL_MASTER   | \
                              INT_EBL_WATCHDOG | \
                              INT_EBL_HEAR_0   | \
                              INT_EBL_HEAR_1   | \
                              INT_EBL_HEAR_2   | \
                              INT_EBL_HEAR_3   | \
                              INT_EBL_HEAR_4   | \
                              INT_EBL_FIFO_A0)
```

7. Check the `X_HEAR_FUNCTION_CHAIN4_DONE` flag in the waitloop in `app.cfs`.

   Add the following code snippet below the code segment that checks and processes the `X_HEAR_FUNCTION_CHAIN3_DONE` flag:

```
// Check the HEAR Function Chain 4 done flag
load x0, x[X_HEAR_FUNCTION_CHAIN4_DONE]
cmp x0, 1 ||| nop
goto HEAR_function_chain4_done if eq
```

   This code snippet checks the `X_HEAR_FUNCTION_CHAIN4_DONE` flag and branches to the core block of code indicated by the label `HEAR_function_chain4_done`, if appropriate.

8. Add the “post-function chain 4 done” core block in `app.cfs`.

   The following is an example of a block of code that could be added:

```
HEAR_function_chain4_done:

    // Clear the flag
    store x[X_HEAR_FUNCTION_CHAIN4_DONE], 0

    // Re-enable interrupts
    Sys_Int_Master_Enable
    // Do whatever needs to be done at this point
    ...

    // Branch back to the top of the main loop
    goto waitloop
```

5 Porting Guide

5.1 INTRODUCTION

This chapter acts as a porting guide for developers familiar with BelaSigna 250 and BelaSigna 200 application development. Where possible, BelaSigna 250 and BelaSigna 200 examples are used as a basis for comparison. This chapter can also serve as a reference for important programming features of BelaSigna 300. As a reference, it can be useful to developers who are not familiar with BelaSigna 250 and BelaSigna 200. The tutorial assumes some familiarity with the ON Semiconductor development tools such as the Integrated Development Environment (IDE) and the appropriate assembly programming toolchains.

The BelaSigna 300 programming model centers around the 24-bit CFX DSP core, which serves as master of the system. The application developer has full control over the programmability of the CFX. As the master, the CFX is responsible for initialization of the IOC, FIFO controller, HEAR, and any other peripherals and external interfaces required by the application.

The HEAR Configurable Accelerator is configured using the HEAR Configuration Tool (HCT) and the resulting microcode is embedded in the CFX program. It provides essential signal processing functionality such as FIR/IIR filtering, vector-based arithmetic instructions, as well as updated versions of the functions available from the WOLA filterbank coprocessor found on BelaSigna 250 and BelaSigna 200.

When comparing BelaSigna 250 and BelaSigna 200 to BelaSigna 300, there is natural transition from the RCore to the CFX and from the WOLA filterbank coprocessor to the HEAR. In the simplest cases, an BelaSigna 250 or BelaSigna 200 application's structure can be ported to BelaSigna 300 by mapping all of the RCore code to equivalent CFX code and mapping the WOLA filterbank coprocessor functions to the HEAR. However, BelaSigna 300 offers significantly more flexibility and functionality, so this type of porting will not necessarily produce an application that is optimized for the BelaSigna 300 system. Ideally, the application's processing will be remapped to a combination of code on the CFX and microcode on the HEAR which produces a good load balance between the two cores.

5.2 DEVELOPMENT TOOLS

5.2.1 Toolchain

The toolchain for BelaSigna 300 is similar to that used for BelaSigna 250 and BelaSigna 200. It consists of a preprocessor, assembler, object librarian, and linker. The program file `cfcc.exe` is the toolchain driver. In addition to these tools, a new separate tool called the HEAR Configuration Tool (HCT) is used for generating application-specific microcode. When using BelaSigna 250 and BelaSigna 200, the WOLA filterbank coprocessor microcode was provided only in pre-compiled form as a library of object code.

The filename extensions used by the DSP core toolchains are summarized in Table 4. This provides a conceptual mapping from typical BelaSigna 250 and BelaSigna 200 development files to

BelaSigna 300 development files. Refer to the *CFX Toolchain Reference* for more information regarding the development tools.

TABLE 4: FILENAME EXTENSIONS USED THE BY THE RCORE AND CFX TOOLCHAIN

Description	Filename extension for RCore toolchain	Filename extension for CFX toolchain
Assembly language source file	.ss	.cfs
Assembly language header file	.inc	.cfi
Microcode configuration file	N/A	.hct
Preprocessed assembly language source file	.s	.cft
Program listing file	.lst	.lst
Object module	.obj	.obj
Object module library	.lib	.lib
Executable output file	.abs	.abs

5.2.2 Integrated Development Environment

The IDE used for BelaSigna 300 development is similar to that used with previous platforms and releases of the Evaluation and Development Kit. To work on an BelaSigna 300 development project in the IDE, we suggest you use SignaKlara Development Projects of the type **BelaSigna 300 Project**.

The IDE's build process is analogous to the build process used for previous platforms. The only addition is the generation of microcode using the HCT. Processing of the HCT input file and subsequent embedding of the microcode into the application is handled as part of the automated build process in the IDE.

For more information on the IDE, consult the *Integrated Development Environment User's Guide*.

5.2.3 Chip IDs

To use the firmware include files, BelaSigna 300 applications must define a preprocessor symbol named `SK3_CID` to identify the target device. This is analogous to the `SK2_CID` symbol that was used for BelaSigna 250 and BelaSigna 200. A BelaSigna 300 chip ID has a two-part structure and thus a different value from those used for BelaSigna 250 or BelaSigna 200. The *Firmware Reference Manual* provides more information about SK3 variants, and how you can determine your device's chip ID.

5.3 CFX PROGRAMMING

This section describes some key architectural differences between the RCore and CFX and how they relate to common programming tasks. This section can be used as a guide for porting RCore code directly to the CFX DSP. In some cases, examples are used to illustrate how these tasks are programmed for the CFX and the corresponding RCore program code is shown for reference.

In the CFX, all data manipulations must be done on data that is first loaded into registers. In the RCore, manipulations were possible in memory directly as well as on data loaded in registers. By restricting direct access to memory from the CFX, the resulting clean interface enabled significant hardware optimization, and hence, execution speed.

For the most common instructions, a suggested mapping from RCore instructions to CFX instructions is provided in this section. These are suggestions only and in many cases there is no instruction equivalence between the RCore and CFX due to their fundamental architectural differences. However, these suggestions can provide a good starting point for RCore developers to become proficient with the CFX instruction set and architecture.

5.3.1 Instruction Types

The CFX DSP uses 32-bit instruction words composed of one or more independent instructions. The sub-instructions which are part of an instruction are analogous to single instructions on the RCore DSP or other processor architectures. The categories of possible instructions include the following:

Single Arithmetic Instructions

These perform arithmetic operations such as addition, logical operators, shifting, multiplication, etc. They use data registers and/or immediate values as sources/destinations.

Dual Arithmetic Instructions

Dual Arithmetic instructions perform two arithmetic instructions in parallel, but each instruction is more restricted in its operands. One of the two instructions works with the X multiplier or ALU, and is called an X Arithmetic instruction. The other instruction works with the Y multiplier or ALU and is called a Y Arithmetic instruction.

Single Move Instructions

Single Move instructions are used to move data between the memory and the CFX registers. They are also used to modify addressing registers.

Dual Move Instructions

Dual Move instructions perform two instructions in parallel that move data between the memory and the CFX's registers. One instruction works with the X memory bus and is called an X Move instruction, while the other works with the Y memory bus and is called a Y Move instruction.

Long Instructions

Long instructions perform actions such as control flow, system control, load/store direct with 16-bit direct addresses, address register modification with 16-bit immediate values, and loading registers with full-sized immediate values.

Instruction words encode one or two instructions in the following combinations:

Arithmetic-Move Instruction Word

An Arithmetic-Move instruction word performs two instructions: one Single or Dual Arithmetic instruction and one Single or Dual Move instruction. The instructions in an Arithmetic-Move instruction are executed in parallel.

Move-Move Instruction Word

The Move-Move instruction word allows you to compactly specify two Move instructions in a single 32-bit instruction. The two Move instructions may in turn contain Single or Dual Move instructions. Four moves are thus possible with one instruction although the execution of a Move-Move instruction takes two cycles.

Long Instruction Word

A Long instruction word is composed of a single Long instruction.

For more information regarding the instruction types, consult the *CFX DSP Architecture Manual*.

5.3.2 Syntax Basics

CFX assembly language uses the same types of comments as RCore assembly language, except that the new CFX assembler (version 3 or greater) does not treat a semicolon as the start of a comment; instead, semicolons separate multiple instructions. Normally, C/C++ style comments are used exclusively.

Optionally, the semicolon (;) character can be used at the end of an instruction, as shown in the following example:

```
// The following instruction performs no operation
nop;
```

The ||| token is used as a connecting symbol to indicate the aggregate Arithmetic-Move instructions, as follows:

```
add a0, a0, x0 ||| load b0, x[xp0]
```

Similarly, the >>> token is used as a connecting symbol for Move-Move instructions:

```
load x0, x[xp0] >>> load x1, x[xp1]
```

When Dual Arithmetic or Dual Move instructions are used, the parallel instructions are separated by the !! token:

```
// Dual Arithmetic instruction
mul a0, x0, y1 !! mul b0, y0, x1 ||| nop
// Dual Move instruction
nop ||| move a1, x0 !! move b1, y0
// Dual Arithmetic and Dual Move instruction
mul a0, x0, y1 !! mul b0, y0, x1 ||| move a1, x0 !! move b1, y0
```

As with BelaSigna 250 and BelaSigna 200, the assembler is not case-sensitive when recognizing instructions. However, by convention, lower-case is normally used for CFX assembly code mnemonics and register operands.

For more information on the CFX assembly language syntax, consult the *CFX Toolchain Reference*.

5.3.3 Delay Slots and Synthetic Instructions

Many CFX instructions that control program flow have *delay slots*—one or two instructions after the control flow instruction which execute before the program flow actually changes to the new location—to avoid stalling the CFX's execution pipeline.

The number of delay slots in a control flow instruction is shown with a `.nd` suffix on the instruction mnemonic, where `n` is 0, 1 or 2. This is the number of *clock cycles*; a single delay slot *must* be filled with one single-cycle instruction, while two delay slots can be filled with either one two-cycle instruction or two single-cycle instructions. There are further restrictions on delay-slot instructions which are detailed in the *CFX DSP Architecture Manual*. The *CFX Toolchain Reference* describes how the CFX assembler checks these restrictions.

While using delay-slot instructions can improve the performance of CFX programs, it is more difficult to verify their logical correctness (due to their somewhat out-of-order execution) and it is complicated to ensure that the delay slots contain the right kinds of instructions. For these reasons, the CFX assembler also provides *no-delay-slot synthetic* (implemented by the assembler, not by the CFX DSP core) versions of the control flow instructions to simplify the instruction set for new developers, and to speed up the initial phase of program development when optimization is not important.

These no-delay-slot instructions have the mnemonics `call`, `goto`, `loop`, `return` and `returni`. Where a zero-delay-slot form exists in the native instruction set (`goto.0d` and `returni.0d`), the no-delay-slot form is a straightforward *synonym* for the machine's `.0d` instruction. Where the native instruction set only has one-delay-slot or two-delay-slot instructions, the no-delay-slot forms are *synthesized* from the machine instruction followed by one or two `nop` cycles.

The BelaSigna 300 sample code applications use the synthetic no-delay-slot instructions for simplicity and brevity in situations where the delay slots cannot be used, or where the code is designed as a template to be filled in (and thus rearranged). The use of these instructions is also illustrated in Section 5.3.7, "Control Flow Instructions" on page 35. See the *CFX Toolchain Reference* for more information about synonyms and synthesized instructions.

5.3.4 Registers

The CFX DSP registers can be grouped into three categories: data registers, addressing registers and program control registers.

Data Registers

The CFX has four accumulators that are each 56 bits wide: `a0`, `a1`, `b0`, `b1`. These accumulators are separated into a low part, a high part, and an extension part, similar to the RCore's accumulator separation into `AL`, `AH`, and `AE`. For example, accumulator `a0` is divided into `a0.l` (low part, 24 bits), `a0.h` (high part, 24 bits), and `a0.e` (extension part, 8 bits). The CFX accumulators are generally used for the same purposes as the RCore accumulator.

Registers `x0`, `x1`, `y0`, and `y1` are used as general-purpose data registers and as inputs to the multipliers. Each of these registers has a width of 24 bits. As inputs to the multipliers, these registers are analogous to the `X` and `Y` registers on the RCore. As general-purpose data registers, they can fill a similar role as `D_AUX_REG0` or `D_AUX_REG4` on the RCore. Unlike the `D_AUX_REG` registers on the RCore, these registers are an integrated part of the datapath and can be used as both destination and source for many arithmetic instructions.

Addressing Registers

The pointer registers are `xp0`, `xp1`, ... `xp7`, and `yp0`, ... `yp3`. Like the RCore's `R0`, `R1`, ... `R7` registers, these contain addresses that are each 16 bits wide. They are used for indirect addressing on the CFX in a manner analogous to the RCore's pointer registers.

Each of the CFX pointer registers has a 16-bit `step` and a 15-bit `circ` register associated with it. For example, `xp0` is associated with the `xstep0` and `xcirc0` registers. The `xstep0` register defines the step-size applied when `xp0` is used in post-modification mode. The `xcirc0` register is used to define special addressing modes including circular buffer addressing and bit-reversed addressing. The combination of the `step` and `circ` registers perform the same role on the CFX that the `PCFG` registers perform on the RCore.

Program Control Registers

The CFX has a dedicated stack pointer called `sp` (equivalent to `R3` and `R7` for RCore), and the status register is `sr` (equivalent to `ST` for the RCore). Both of these registers are 16 bits wide.

Like the RCore, the CFX has a 16-bit program counter (`pc`). The CFX also has a set of control registers for handling function calls and interrupt service routines (ISRs). These registers are:

lr link register, 16 bits, used to store the return address during a subroutine

ilpc interrupt link program counter, 16 bits, used to store the return address during an interrupt service routine

ilsr interrupt link status register, 16 bits, used to store the status register during an interrupt service routine

Since the CFX link registers need not be saved unless nested subroutines or interrupts are expected, execution speed (particularly for short routines) is enhanced.

There are no equivalents for these three registers on the RCore, since the RCore uses the stack for storing the program counter during function calls or ISRs.

Three registers are associated with the loop stack: `lc` (loop counter, 12 bits), `lsa` (loop start address, 16 bits), and `lea` (loop end address, 24 bits). The `lc` register is analogous to the `LC0` and `LC1` loop counter registers on the RCore; however, there is no equivalent to the `lsa` and `lea` registers since the RCore does not have zero overhead loop branching. You will not normally access these registers directly. Instead, use the zero-overhead loop instructions described in Section 5.3.5, “Loops”.

5.3.5 Loops

The CFX uses a hardware loop stack to implement efficient hardware looping. Up to four nested hardware loops are supported.

When a loop is started, the loop stack is pushed with the following data:

- loop start address
- loop end address
- number of times the loop must be executed (1 to 4096)

The instructions that start loops on the CFX have two delay slots. The instructions in the delay slots execute once at the start of the loop, but are not executed during each iteration.

The loop mechanism on the CFX performs zero-overhead looping by tracking the current `pc` value as compared to the loop end address. Thus, there is normally a label for the end of the loop and not the beginning of the loop in CFX code. This contrasts with an RCore loop, where there is normally a label only for the beginning of the loop.

Because of this zero-overhead looping mechanism, the end address for nested loops cannot be the same. In the following comparison between RCore loops and CFX loops, a `nop` instruction is inserted to avoid this situation.

BelaSigna 250 and BelaSigna 200 code:

```
    LDSI LC0, (LOOP_COUNT_1 - 1)
loop_label_1:
        // Outer loop body...
        LDSI LC1, (LOOP_COUNT_2 - 1)
loop_label_2:
            // Inner loop body...
        DBNZ1 loop_label_2
    DBNZ0 loop_label_1
```

BelaSigna 300 code:

```
    loop.2d LOOP_COUNT_1, loop_label_1
    // Delay slots 1 and 2
    nop >>> nop
        // Outer loop body...
        loop.2d LOOP_COUNT_2, loop_label_2
        // Delay slots 1 and 2
        nop >>> nop
            // Inner loop body...
loop_label_2:
            nop
loop_label_1:
        nop
```

The immediate value used to specify the number of iterations of a CFX hardware loop corresponds exactly to the number of desired iterations, except for the special case of 0 which corresponds to 4096 iterations. In contrast, the value loaded into the `LC0` or `LC1` register of the RCore corresponds to N-1, where N is the number of desired iterations. This difference can be seen in the immediate values used for the `LDSI` instructions in the example above. As with the RCore, it is also possible to start a loop on the CFX using a data register as the source for the loop count.

The size of the loop counter is also different between the CFX and the RCore. The CFX loop counters are 12 bits wide, whereas the RCore loop counters for BelaSigna 250 are 16 bits wide, and the loop counters for BelaSigna 200 are only 8 bits wide.

The hardware loop stack of the CFX has four levels of nesting available. Thus, a maximum of four nested hardware loops can be used at any given time on the CFX DSP. Starting another loop when the loop stack is full has no effect; therefore, during the initialization phase of an application, we recommend popping any values that might have remained from the execution of previous program code. One set of values is popped from the loop stack by an explicit read of the `lea` register or by using the `endloop` instruction.

Be aware that there are restrictions on which instructions can be used as the last two or three instructions in a hardware loop. The most commonly encountered of these restrictions include the following:

- The last two or three instructions of the loop cannot contain certain control-flow related instructions: `call`, `endloop`, `goto`, `interrupt`, `return`, `returni`.
- If the last instruction of the loop involves a multiplication, you must be careful with multiplication restrictions on both the first instruction of the loop body and the instruction after the loop.

There are additional restrictions involving more complicated or uncommon examples.

For a complete discussion of the instructions you are allowed to use near the end of a hardware loop, refer to the *CFX DSP Architecture Manual*. For information about the CFX assembler's restriction checking, see the *CFX Toolchain Reference*.

5.3.6 Stack Usage

A CFX application has one single stack that resides in X memory, in contrast to the independent system and user stacks available on the RCore. The following examples show how the stack is pushed and popped. The standard macros for pushing and popping groups of registers (the example uses groups of three registers, but similar macros are available for different numbers of registers) are recommended in most cases.

BelaSigna 250 and BelaSigna 200 code:

```
// Equivalent to the following instructions:
//      PUSH AE
//      PUSH AH
//      PUSH AL
// Stack pointer decremented by 3 words
PUSH_3(AE, AH, AL)

// Equivalent to the following instructions:
//      POP AL
//      POP AH
//      POP AE
// Stack pointer incremented by 3 words
POP_3(AE, AH, AL)
```

BelaSigna 300 code:

```
// Equivalent to the following instructions:
//      add sp, 3 >>> store x[sp-1], a0.e
//      store x[sp-2], a0.h >>> store x[sp-3], a0.l
// Stack pointer incremented by 3 words
Sys_Push_Reg_3(a0.e, a0.h, a0.l)

// Equivalent to the following instructions:
//      load a0.e, x[sp-1] >>> load a0.h, x[sp-2]
//      load a0.l, x[sp-3] >>> add sp, -3
// Stack pointer decremented by 3 words
Sys_Pop_Reg_3(a0.e, a0.h, a0.l)
```

Although the standard macros from the RCore and CFX provide similar functionality, some differences exist at the instruction level. The CFX stack grows to increasing addresses, as compared to the RCore stack which grows downward to decreasing addresses. On the CFX, one cycle is required to explicitly modify the stack pointer, and the stack is accessed using offset addressing. This offset addressing can be useful for accessing arbitrary values within the stack without modifying the stack pointer, which is a feature not available on the RCore.

In program code where useful arithmetic instructions can be executed in parallel with the move instructions associated with the stack usage, it is recommended that Arithmetic-Move instructions be used instead of the Move-Move instructions resulting from the expansion of the standard macros.

5.3.6.1 Stack Initialization

The initialization of the RCore's stack pointers is typically done in the following way:

```
LDI R3, SYS_STACK_TOP
LDI R7, USR_STACK_TOP
```

In this example, the constants SYS_STACK_TOP and USR_STACK_TOP are user-defined values for the top of the system and user stacks respectively. For the CFX, the corresponding stack initialization code is as follows:

```
load sp, STACK_BOTTOM
```

The STACK_BOTTOM constant in the example is user-defined. Any valid address in X memory can be used to specify the location of the bottom of the stack; however, it is recommended that the stack be placed at the high part of X memory, for example starting at address 0x1780.

5.3.7 Control Flow Instructions

Control flow instructions (i.e., calls, branches, returns, etc.) on the CFX are generally more complicated than on the RCore due to the use of delay slots. Instructions in the one or two delay slots following control flow instructions are executed before performing the control flow action. Control flow instructions on the RCore do not have delay slots. For more information about the CFX's delay slots, see Section 5.3.3, "Delay Slots and Synthetic Instructions" on page 30.

Subroutines

The call and return instruction types are used to implement subroutines on the CFX DSP. These are analogous to the CALL and RET instructions on the RCore; however the CFX instructions come in several different types with different numbers of delay slots.

When a variant of the call instruction is executed, the return address (i.e., the address of the instruction following the instruction(s) in the delay slot(s)) is stored in the link register lr. There is only a single link register; therefore you are responsible for saving and storing its contents to allow nested subroutines. The example code below demonstrates this manipulation of the link register, and compares a typical RCore CALL instruction to various forms of the CFX call instruction.

BelaSigna 250 and BelaSigna 200 code:

```
// Call direct
CALL FUNCTION_LABEL
// Some other code...


FUNCTION_LABEL:
    // The function code goes here...

    // return to calling code
    RET
```

BelaSigna 300 code:

```
// Save the link register
Sys_Push_Reg_1(lr)

// Call direct with one delay slot
call.1d FUNCTION_LABEL
nop                         // Delay slot

// Call direct - synthesized with hidden nop in delay slot
call FUNCTION_LABEL

// Call indirect with two delay slots
load xp0, FUNCTION_LABEL
call.2d xp0
nop                         // Delay slot 1
nop                         // Delay slot 2
```

```
// Restore the link register
Sys_Pop_Reg_1(lr)

// Some other code...

FUNCTION_LABEL:
    // The function code goes here...

    // Return with two delay slots
    return.2d
    nop >>> nop            // Delay slots 1 and 2

    // Or, return - synthesized with two hidden nop cycles
    //              in the delay slots: nop >>> nop
    return
```

Branches

The `goto` instruction type is used on the CFX in the same manner as the `BRA` instruction on the RCore. The `goto` instruction changes the program counter to the address specified either as a direct address or in an `xp` register. Conditional `goto` instructions forms are also available, in a similar manner to the conditional `BRA` instruction on the RCore. An example of some `goto` instructions as compared to the typical RCore `BRA` instruction is given below:

BelaSigna 250 and BelaSigna 200 code:

```
// Branch direct
BRA BRANCH_LABEL

// Conditional branch direct
BRA TRUE_LABEL, Z
```

BelaSigna 300 code:

```
// Branch direct with no delay slot
goto.0d BRANCH_LABEL

// Branch direct - synonym with no delay slot
goto BRANCH_LABEL

// Branch direct with one delay slot
goto.1d BRANCH_LABEL
nop                         // Delay slot

// Conditional branch direct with no delay slot
goto.0d TRUE_LABEL, zero

// Conditional branch direct - synonym with no delay slot
//                             and alternate conditional form
goto TRUE_LABEL if zero

// Conditional branch direct with two delay slots
goto.2d TRUE_LABEL, zero
nop >>> nop                 // Delay slots 1 and 2
```

5.3.8 Arithmetic Instructions

This section describes the basic CFX arithmetic and logical instructions, specifically those which do not involve the use of the multipliers. Multiplies and related instructions are covered in Section 5.3.9, “Multiplication Instructions”.

Arithmetic instructions on the CFX always operate on the datapath registers - `a0`, `b0`, `a1`, `b1`, `x0`, `x1`, `y0`, and `y1`. They are generally analogous to instructions on the RCore which operate directly on the accumulator. Single Arithmetic instructions can operate on any of the data registers.The

Dual Arithmetic instructions (i.e., X Arithmetic and Y Arithmetic) have limited operands. A greater variety of X Arithmetic instructions are available.

The CFX arithmetic instructions are always performed with 56 bits, with the input and output precision being determined by the source and destination registers. Arithmetic instructions update the flags for the associated ALU in the CFX DSP's status register (`sr`). The CFX also has a number of different arithmetic instruction forms for working with fractional, signed, unsigned, and single- or double-precision data. For more information on these details consult the *CFX DSP Architecture Manual*.

Some suggested mappings between the RCore arithmetic/logical instructions and CFX arithmetic instructions are given in Table 5 and Table 6. These are suggestions only and do not necessarily represent the only way of performing the equivalent instruction. They represent a starting point for someone who is already familiar with the RCore DSP and is moving to the CFX DSP. In some cases, there is no direct equivalent, or the equivalent CFX instruction is actually a Move instruction because the instruction involves address registers. For these cases, consult Section 5.3.11, "Data Movement Instructions".

The CFX DSP also provides a number of arithmetic instructions which do not have equivalents on the RCore DSP. These include addition with right shift, subtraction with right shift, minimum of two registers, and maximum of two registers. Furthermore, a number of alternate instruction forms exist that are particular to unsigned (i.e., no extension or `.nx`) and double width (`.dw`) computations. Some of these instructions are also specified in Table 5 and Table 6.

TABLE 5: SUGGESTED ARITHMETIC INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments
`ABS A [,Cond] [,DW]`	`abs d-reg, d-reg`	An X Arithmetic instruction is also available on the CFX, for which the input must be an accumulator and the output must be either `a0` or `a1`. A conditional absolute value instruction is not available.
`ADD A, Reg [,C]`	`add d-reg, d-reg, d-reg`	Dual-Arithmetic forms of this instruction are also available which work only on accumulator registers.
`ADD A, Rc [,C]`	N/A	An address register on the CFX cannot be used as an operand to an arithmetic instruction. An equivalent Move instruction can usually be used.

`ADD A, (Rij) [,C]`
`ADD A, (Rij)p [,C]`
`ADD A, DRAM [,B]`
	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

`ADDI A, IMM [,C]` `ADSI A, SIMM10`	`add d-reg, d-reg, imm3s` `add.fr d-reg, d-reg, imm3s`	The CFX does not have any additive instructions with full-sized immediates. The `add.fr` form can be used to add special fractional immediate values corresponding to -1, -3/4, -1/2, -1/4, +1/4, +1/2, +3/4 and +1.
`CLB A`	`log2abs d-reg, d-reg` `nlog2abs d-reg, d-reg`	The `nlog2abs` instruction also has an X Arithmetic instruction form.

TABLE 5: SUGGESTED ARITHMETIC INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS (CONTINUED)

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments
`CLR A [,DW]` `CLR Reg`	`load d-reg, imm6s` `load.dw d-reg, imm6s`	Registers can also be initialized with load immediate Long or Move instructions.

`DEC A [,Cond] [,DW]`
`DEC Reg [,Cond]`
	`add d-reg, imm3s, cond3`
	`add d-reg, d-reg, imm3s`
	`add.dw d-reg, imm3s, cond3`
	`add.dw d-reg, d-reg, imm3s`
		The CFX Arithmetic instructions only work with data registers. To decrement an address register use the equivalent Move instruction.

`DEC (Rij) [,Cond]`	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

`INC A [,Cond] [,DW]`
`INC Reg [,Cond]`
	`add d-reg, imm3s, cond3`
	`add d-reg, d-reg, imm3s`
	`add.dw d-reg, imm3s, cond3`
	`add.dw d-reg, d-reg, imm3s`
		The CFX Arithmetic instructions only work with data registers. To decrement an address register use the equivalent Move instruction.

`INC (Rij) [,Cond]`	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.
`NEG A [,Cond] [,DW]`	`neg d-reg, d-reg` `neg d-reg, d-reg, cond3`	An unconditional X Arithmetic instruction is also available on the CFX, for which the input must be an accumulator and the output must be either `a0` or `a1`.
`RND A`	N/A	Rounding is done automatically on the CFX as part of certain Long and Move instructions involving accumulators. See Section 5.3.12, “Rounding and Saturation”.

`SHFT SIMM6`
`SHFT A [,Cond] [,INV]`
	`shra d-reg, imm6u`
	`shra d-reg, d-reg, imm3u`
	`shra d-reg, d-reg, cond3`
	`shra d-reg, imm3u, cond3`
		A number of other logical right shift (`shrl`) and logical left shift (`shll`) instructions are available on the CFX. The logical left shift instructions can usually be used when the `INV` option would be used on the RCore.

`SUB A, Reg [,C]`	`sub d-reg, d-reg, d-reg`	Dual-Arithmetic forms of this instruction are also available which work only on accumulator registers.
`SUB A, Rc [,C]`	N/A	An address register on the CFX cannot be used as an operand to an arithmetic instruction. An equivalent Move instruction can usually be used.

`SUB A, (Rij) [,C]`
`SUB A, (Rij)p [,C]`
`SUB A, DRAM [,B]`
	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

TABLE 5: SUGGESTED ARITHMETIC INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS (CONTINUED)

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments
`SUBI A, IMM [,C]` `SUSI A, SIMM10`	`add d-reg, d-reg, imm3s` `add.fr d-reg, d-reg, imm3s`	The CFX does not have any additive instructions with full-sized immediates. A negative signed immediate in the `add` and `add.fr` instructions can be used to implement equivalent immediate subtractions for limited immediate values.
N/A	`addsh d-reg, d-reg, d-reg` `subsh d-reg, d-reg, d-reg`	These perform an addition or subtraction with post right-shift. There are no single-instruction equivalents on the RCore. The CFX also has X and Y Arithmetic forms of these instructions which are limited to working on the accumulators.
N/A	`add.nx d-reg, d-reg, d-reg` `sub.nx d-reg, d-reg, d-reg`	These perform an addition or subtraction without using extension bits, which can be interpreted as an unsigned operation. There are no equivalent instructions on the RCore. The CFX also has X Arithmetic forms of these instructions.
N/A	`max d-reg, d-reg, d-reg` `min d-reg, d-reg, d-reg`	These select the maximum or minimum value from two registers. There are no equivalent instructions on the RCore. The CFX also has X Arithmetic forms of these instructions.

N/A	`shll.nx d-reg, d-reg, d-reg`
	`shra.nx d-reg, d-reg, d-reg`
	`shrl.nx d-reg, d-reg, d-reg`
		These perform logical left, arithmetic right and logical right shifts of data registers without using extension bits. There are no equivalent instructions on the RCore. The CFX also has additional forms of these instructions which use various sizes of immediate shift counts.

TABLE 6: SUGGESTED LOGICAL INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments
`AND A, Reg`	`and d-reg, d-reg, d-reg`	An X Arithmetic form of this instruction is also available.
`AND A, Rc`	N/A	An address register on the CFX cannot be used as an operand to an arithmetic instruction.

`AND A, (Rij)`
`AND A, (Rij)p`
`AND A, DRAM [,B]`
	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

`ANDI A, IMM` `ANSI A, SIMM10`	`and d-reg, d-reg, imm3s` `and.dw d-reg, d-reg, imm3s`	The CFX does not have any arithmetic instructions with full-sized immediates.
`CMPL A [,Cond] [,DW]`	N/A	To implement a complement bits instruction, use an `xor` instruction with -1.

TABLE 6: SUGGESTED LOGICAL INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS (CONTINUED)

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments
`EOR A, Reg`	`xor d-reg, d-reg, d-reg`	None
`EOR A, Rc`	N/A	An address register on the CFX cannot be used as an operand to an arithmetic instruction.

`EOR A, (Rij)`
`EOR A, (Rij)p`
`EOR A, DRAM [,B]`
	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

`EORI A, IMM` `EOSI A, SIMM10`	`xor d-reg, d-reg, imm3s` `xor.dw d-reg, d-reg, imm3s`	The CFX does not have any arithmetic instructions with full-sized immediates.
`OR A, Reg`	`or d-reg, d-reg, d-reg`	An X Arithmetic form of this instruction is also available.
`OR A, Rc`	N/A	An address register on the CFX cannot be used as an operand to an arithmetic instruction.

`OR A, (Rij)`
`OR A, (Rij)p`
`OR A, DRAM [,B]`
	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

`ORI A, IMM` `ORSI A, SIMM10`	`or d-reg, d-reg, imm3s` `or.dw d-reg, d-reg, imm3s`	The CFX does not have any arithmetic instructions with full-sized immediates.
`RES Reg, Bit`	`bitclr d-reg, bit-index`	The CFX arithmetic instruction only works with data registers.
`RES (Rij) Bit`	N/A	An indirect memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.
`SET Reg, Bit`	`bitset d-reg, bit-index`	The CFX arithmetic instruction only works with data registers.
`SET (Rij) Bit`	N/A	An indirect memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.
`TGL Reg, Bit`	`bittgl d-reg, bit-index`	The CFX arithmetic instruction only works with data registers.
`TGL (Rij) Bit`	N/A	An indirect memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

5.3.9 Multiplication Instructions

This section discusses the use of CFX multiplicative instructions and how they relate to setting up a typical multiply-accumulate (MAC) sequence. Any multiply on the CFX is a two-cycle instruction, where a multiplier computation is performed during the first cycle and then an ALU computation is performed during the next cycle. The two instructions can be pipelined such that a MAC

instruction can be started during the second cycle of a previous MAC instruction. This makes it possible to complete K MAC instructions in K+1 clock cycles. This two-cycle MAC system is similar to, but not exactly the same as, the pipelined MAC system on the RCore DSP.

The CFX has two multipliers and two ALUs that can be utilized in parallel through the use of Dual Arithmetic instructions. The result of one multiplier is written to either accumulator `a0` or `a1`, while the result of the other multiplier is written to accumulator `b0` or `b1`.

The following example shows implementations of typical MAC loops on the RCore and the CFX.

BelaSigna 250 and BelaSigna 200 code:

```
//
// Multiply-accumulate of length MAC_LENGTH
//
// Load first pair of operands
MSET (R4+), (R0+)
// Load second pair of operands, clear accumulator, update product
MLD (R4+), (R0+)
// Load the remaining operands while accumulating multiplier result
REP (MAC_LENGTH - 2 - 1)
    MPYA (R4+), (R0+)
// Flush pipeline
DADD P
DADD
```

BelaSigna 300 code:

```
//
// Multiply-accumulate of length MAC_LENGTH; uses one
// multiplier
//
loop.2d MAC_LENGTH - 2, loop_label_1
// Delay slot 1: preload operands
nop ||| load x0, x[xp0++] !! load y0, y[yp0++]
// Delay slot 2: first multiply and load next operands
mul a0, x0, y0 ||| load x0, x[xp0++] !! load y0, y[yp0++]
loop_label_1:
    // Multiply-accumulate, and load next operands
    muladd a0, x0, y0 ||| load x0, x[xp0++] !! load y0, y[yp0++]
// Final multiply-accumulate iteration (two cycles)
muladd a0, x0, y0 ||| nop
nop
```

5.3.9.1 Multiplication Instruction Mappings

Some suggested mappings between the RCore multiplication instructions and CFX arithmetic instructions are given in Table 7. The differences in the multiplier pipeline and architecture between the two DSPs means that there are no direct equivalents between the two cores. The load/store architecture of the CFX means that the multiplication and memory load instructions must be pipelined and explicitly done in parallel to create an equivalent MAC sequence.

The CFX DSP also provides a number of arithmetic instructions for multiplication which do not have equivalents on the RCore DSP. These include multiply with negation, multiplication with pre-addition or subtraction (using the X multiplier's pre-adder), and explicit signed-unsigned (`.su`) or unsigned-unsigned (`.uu`) multiplications. The most commonly used instructions are listed in Table 7.

TABLE 7: SUGGESTED MULTIPLICATION INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments
`DADD [Cond] [,P]`	`add d-reg, d-reg, d-reg` `add d-reg, d-reg, cond3`	Any addition instruction between accumulators can be used on the CFX to implement a full-precision add.
`DSUB [Cond] [,P]`	`sub d-reg, d-reg, d-reg` `sub d-reg, d-reg, cond3`	Any subtraction instruction between accumulators can be used on the CFX to implement a full-precision subtract.
`MUL [Cond] [,A] [,P]`	`mul ab-reg, xy-reg, xy-reg`	Dual Arithmetic versions of the `mul` instruction with greater limitations on inputs and outputs are also available. The RCore instruction is used after the multiplication has already started by loading the `X` or `Y` registers, whereas the CFX instruction begins a two-cycle multiply with the result being placed into the accumulator register provided.

`MLD (Rj), (Ri) [,SQ]`
`MLD Reg, (Ri) [,SQ]`
`MSET (Rj), (Ri) [,SQ]`
`MSET Reg, (Ri) [,SQ]`
	N/A	There is no equivalent CFX arithmetic instruction for these multiplier load instructions due to architectural differences in the multiplication hardware.

`MPYA (Rj), (Ri), [,SQ]` `MPYA Reg, (Ri), [,SQ]`	`muladd ab-reg, xy-reg, xy-reg`	Dual Arithmetic versions of the `muladd` instruction with greater limitations on inputs and outputs are also available. The RCore instruction is used to simultaneously load `X` and `Y` registers while performing an addition with the previous product, whereas the CFX instruction begins a two-cycle multiply-add with the result being placed in the accumulator register provided.
`MPYS (Rj), (Ri), [,SQ]` `MPYS Reg, (Ri), [,SQ]`	`mulsub ab-reg, xy-reg, xy-reg`	Dual Arithmetic versions of the `mulsub` instruction with greater limitations on inputs and outputs are also available. The RCore instruction is used to simultaneously load `X` and `Y` registers while performing a subtraction with the previous product, whereas the CFX instruction begins a two-cycle multiply-subtract with the result being placed in the accumulator register provided.
N/A	`mulneg ab-reg, xy-reg, xy-reg`	This instruction performs a negated multiplication on the CFX. There is no equivalent on the RCore. Dual Arithmetic versions of this instruction are also available.

TABLE 7: SUGGESTED MULTIPLICATION INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS (CONTINUED)

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments

N/A	`addmul ab-reg, xy-reg, xy-reg, xy-reg`
	`addmuladd ab-reg, xy-reg, xy-reg, xy-reg`
	`addmulneg ab-reg, xy-reg, xy-reg, xy-reg`
	`addmulsub ab-reg, xy-reg, xy-reg, xy-reg`
	`submul ab-reg, xy-reg, xy-reg, xy-reg`
	`submuladd ab-reg, xy-reg, xy-reg, xy-reg`
	`submulneg ab-reg, xy-reg, xy-reg, xy-reg`
	`submulsub ab-reg, xy-reg, xy-reg, xy-reg`
		These instructions perform multiplication, multiply-adds, multiplication with negation and multiply-subtracts with pre-addition or pre-subtraction of the last two operands.
		These instructions have no equivalents on the RCore.

N/A	`mul.su ab-reg, xy-reg, xy-reg`
	`mulneg.su ab-reg, xy-reg, xy-reg`
	`muladd.su ab-reg, xy-reg, xy-reg`
	`mulsub.su ab-reg, xy-reg, xy-reg`
		These instructions perform multiplier operations while treating one operand as signed and the other as unsigned.
		These instructions have no equivalents on the RCore.

All multiplication instructions, when ST(SM) = 1.	`mul.uu ab-reg, xy-reg, xy-reg`
	`mulneg.uu ab-reg, xy-reg, xy-reg`
	`muladd.uu ab-reg, xy-reg, xy-reg`
	`mulsub.uu ab-reg, xy-reg, xy-reg`
		These instructions perform multiplier operations while treating both operands as unsigned.
		The RCore performs unsigned integer multiplications when the Signed Mode bit in the status register `ST` is set to 1.

5.3.9.2 Multiplication Restrictions

Multiplication instructions have a two-cycle duration. In the first cycle, the X or Y multiplier is in use, and during the second cycle the X or Y ALU is in use. To avoid undefined behavior, refer to the *CFX DSP Architecture Manual* for more information about the restrictions that apply to multiplication instructions. For information about the CFX assembler's restriction checking, see the *CFX Toolchain Reference*.

Examples of situations where undefined behavior can occur include the following:

ALU Resource Contention

ALU resource contention occurs when an instruction which executes during the second cycle of a multiplication instruction uses an ALU that is also used by the multiplication. In the following example, the addition

instruction attempts to use the X ALU during the second cycle of the multiplication instruction, which is unavailable, producing undefined results for both instructions.

```
muladd a0, x0, y0 ||| nop
add x0, x1, b0 ||| nop          // illegal - XALU already in use
```

Register File Resource Contention

Register file resource contention occurs because the registers are organized into register files. For example, the B register file contains the registers `b0` and `b1`. If a register file is used in the second cycle of a multiplication instruction, the other register in the file cannot be accessed in some Arithmetic instructions.

```
add a0, a0, a1 !! muladd b0, y0, y1 ||| nop
// Illegal - B register file containing b0 and b1 is being used due to
// muladd; b1 cannot be read for addition
add x1, b1, y1 ||| nop
```

Reading Outputs—One-Cycle Delay

Because the multiplication instruction takes two cycles, reading the destination register without waiting one cycle results in retrieving the old value of the register, as shown in the example below.

```
muladd a0, x0, x1 ||| nop
// muladd result is not ready; old value of a0 is moved to b0
nop ||| move b0, a0
```

It is also dangerous to rely on the old value being present in the next cycle, because an interrupt might occur after the multiplication starts. In that situation, the multiplication does complete before the interrupt, and hence before the next instruction is executed.

Two Writes Conflict

A conflict occurs when the results of two instructions have the same destination register on the same cycle. In the example below, the behavior is undefined.

```
muladd a0, y0, x0 ||| nop
// Result in a0 is undefined because muladd and
// move instructions both try to write to a0
nop ||| move a0, b0
```

5.3.10 Comparison Instructions and Condition Codes

The CFX arithmetic instructions include various options for performing comparisons using immediate data and registers. As on the RCore, the comparison instructions are like their corresponding subtraction instruction, except that the result of the subtraction is not written to a destination register and only the corresponding ALU flags are updated.

Table 8 provides a suggested mapping between RCore comparison instructions and corresponding CFX arithmetic instructions.

The CFX DSP also supports unsigned comparison (`cmpu.nx`) versions of the standard `cmp` instruction which have no real equivalents in the RCore instruction set. They are listed at the bottom of Table 8.

TABLE 8: SUGGESTED COMPARISON INSTRUCTION TO ARITHMETIC INSTRUCTION MAPPINGS

RCore Instruction(s)	CFX Arithmetic Instruction(s)	Comments
`CMP A, Reg [,C]`	`cmp d-reg, d-reg`	An X Arithmetic form of this instruction is also available.
`CMP A, Rc [,C]`	N/A	An address register on the CFX cannot be used as an operand to an arithmetic instruction.

`CMP A, (Rij) [,C]`
`CMP A, (Rij)p [,C]`
`CMP A, DRAM [,B]`
	N/A	An indirect or direct memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

`CMPI A, IMM [,C]` `CMSI A, SIMM10`	`cmp d-reg, imm3s` `cmp.dw d-reg, imm3s`	The CFX does not have any arithmetic instructions with full-sized immediates.
`DCMP`	`cmp d-reg, d-reg`	To perform a full-precision compare, simply specify two accumulators as the source registers for the `cmp` instruction.
`TST Reg, Bit`	`bittst d-reg, bit-index`	The CFX arithmetic instruction only works with data registers. The CFX puts the result in the Carry flag instead of the Zero flag.
`TST (Rij) Bit`	N/A	An indirect memory reference cannot be used as an operand to an arithmetic instruction. The data in memory must first be loaded into a data register.

N/A	`cmpu.nx d-reg, d-reg`
	`cmpu.nx d-reg, imm3s`
	`cmpu.nx.dw d-reg, imm3s`
		These instructions perform an unsigned comparison and have no equivalents on the RCore.

The CFX instructions with conditional execution support either a 5-bit condition code or a 3-bit condition code, unlike the RCore conditional instructions which always support all of the possible conditions. The 3-bit condition code is more common for arithmetic instructions and covers most of the typical arithmetic comparison results such as “less than”, “greater than”, “zero”, etc. Table 9 provides a comparison between the condition names on the RCore and the CFX for the most typically used conditions. In particular, note that the CFX supports some conditions that are not directly supported on the RCore, such as a strictly positive (i.e., “greater than zero”) condition.

TABLE 9: COMPARISON OF RCORE AND CFX CONDITION NAMES

Condition	RCore Condition Name	CFX Condition Name
zero	`Z`	`zero`
less than zero	`NEG`	`lt`
greater than zero	N/A	`gt`
less than or equal to zero	N/A	`le`
greater than or equal to zero	`POS`	`ge`
carry occurred	`C`	`carry`

TABLE 9: COMPARISON OF RCORE AND CFX CONDITION NAMES (CONTINUED)

Condition	RCore Condition Name	CFX Condition Name
overflow occurred	V	oflow
accumulator extension is in use	E	exten
limiting has occurred	L	sat
address wrap-around has occurred	RiC	N/A
not zero	NZ	not_zero
no carry occurred	NC	no_carry
no overflow occurred	NV	no_oflow
accumulator extension in not use	NE	no_exten
limiting has not occurred	NL	no_sat
address wrap-around has not occurred	NRiC	N/A

5.3.11 Data Movement Instructions

Movement of data on the CFX involves loading or storing from memory as well as performing register-to-register moves. In general, memory accesses can be done via the X, Y or P memory buses. When using a Dual Move instruction, one part of the instruction is performed on the X memory bus and the other part is performed on the Y memory bus.

Data movement instructions can result in data conversions depending on the bit width of the source and destination. Most of these conversions are straightforward (e.g. sign extending or zero extending); however data moves with an accumulator as a source involve rounding and/or saturation. This situation is described in Section 5.3.12, “Rounding and Saturation”.

An important feature of the CFX data movement instructions is the asymmetry between X memory bus accesses and Y memory bus accesses. X memory access is more flexible in terms of source and destination registers than Y memory access. This is another key difference between the CFX and RCore architectures.

The CFX data movement instructions are split between Move instructions and Long instructions. The CFX instruction set mnemonics also make a distinction between *load* (memory to register), *store* (register to memory) and *move* (register to register) instructions. On the RCore, only a single mnemonic (LD) is used for nearly all data movement instructions. Table 10 provides some suggested mappings between common RCore instructions and corresponding CFX instructions. A notable omission from this table is stack-related data movement instructions. For examples of these instructions, see Section 5.3.6, “Stack Usage”.

TABLE 10: SUGGESTED DATA MOVEMENT INSTRUCTION TO CFX INSTRUCTION MAPPINGS

RCore Instruction(s)	CFX Instruction(s)	Comments
LD A, DRAM, B LD DRAM, A, B	load reg, X[imm16u] load reg, Y[imm16u] store X[imm16u], reg store Y[imm16u], reg	These CFX Long instructions allow direct addressing of any address in X or Y memory.
LDSI A, SIMM10 LDI Reg, IMM LDI Rc, IMM LDSI Rn, UIMM10 LDSI LCn, UIMM8	load reg, imm20s load d-reg, imm24s load d-reg, imm5s load.dw d-reg, imm5s load xstep-reg, imm5s load ystep-reg, imm5s load ax-reg, imm2s load by-reg, imm2s	The various immediate CFX instructions are a mixture of Long and Move instructions. Normally, one of these can be used to load the desired register. Those using larger or full-sized immediate values are always Long instructions. Additional immediate load instructions are available; see the *CFX DSP Architecture Manual* for a complete description.
LDI (Rij), IMM	CLR X[single-mode(X)]	The CFX can only initialize an indirectly-addressed memory location with zero, and only in X memory.
LD Reg, Reg LD Reg, Rc LD Rc, Reg LD Rc, Rc	move reg, reg	This Move instruction can be used to transfer data between any two registers (data, address, etc.). Specific X Move and Y Move instructions are also available for limited transfers between specific sets of data registers.
LD Reg, (Rij) LD Rc (Rij) LD (Rij), Reg LD (Rij), Rc	load xmem-reg, X[single-mode(X)] load by-reg, Y[single-mode(Y)] store X[single-mode(X)], xmem-reg store Y[single-mode(Y)], by-reg	These indirect memory access instructions can use any of the addressing modes as described in Section 5.3.11.2, “Address Arithmetic and Addressing Modes”. Specific X Move and Y Move instructions are also available for more limited memory access between specific sets of data registers.
LD Reg, (Reg)p LD Reg, (Rij)p LD (Rij)p, Reg	load reg, P[single-mode(X)] load reg, P[single-mode(Y)] load.ex ab-reg, P[single-mode(X)] load.ex ab-reg, P[single-mode(Y)] store P[single-mode(X)], reg store P[single-mode(Y)], reg store.ex P[single-mode(X)], ab-reg store.ex P[single-mode(Y)], ab-reg	In general, indirect P memory access can be done using an X or Y memory bus pointer register. The CFX cannot use a data register as an indirect address.
N/A	load reg, P[imm16u] load.ex ab-reg, P[imm16u] store P[imm16u], reg store.ex P[imm16u], ab-reg	P memory can also be accessed using load and store direct instructions in the same manner as X and Y memory.

5.3.11.1 Other Data Movement Instructions

The CFX has a number of other data movement instructions which either do not have an RCore equivalent, or require more discussion than those described in Table 10. These other instructions include the following:

Store Immediate Direct Instructions

The CFX DSP has store immediate direct instructions for storing 8-bit signed values to any memory location in X, Y, XY or P memory. This can be useful for loading small values directly into memory. In particular, these instructions can be used for setting small control registers and some control registers on BelaSigna 300 are mapped into 8-bit chunks specifically for this purpose.

```
// Copy the flag directly into an X memory location.
store x[X_FLAG], 0x1
```

Double-width Load and Store from XY Memory

The CFX can access X and Y memory in an interleaved fashion as double-width, 48-bit data. There are Long instructions for accomplishing this with direct (i.e., immediate) addresses and Move instructions for indirect access via an X memory pointer register.

```
// Load/store 48-bit values using XY memory and a direct address
load.dw a0, xy[XY_DATA]
store.dw xy[XY_DATA], b1

// Load/store 48-bit values using XY memory and a pointer register
nop ||| load.dw a0, xy[xp0++]
nop ||| store.dw xy[xp1++], b1
```

Pipelined Memory Copy with Dual-Move Instructions

The RCore for BelaSigna 250 supports a pipelined memory copy using the `LD (Rj), (Ri)` or `LD (Ri), (Rj)` instructions. An example of using this feature is given below:

```
// Copy NUM_WORDS from memory pointed to by R0
// to memory pointed to by R4. This uses D_AUX_REG0
// as intermediate storage in a pipelined copy.
LD D_AUX_REG0, (R0+)
REP (NUM_WORDS - 2)
    LD (R4+), (R0+)
LD (R4+), D_AUX_REG0
```

It is possible to create a similar pipelined copy using Dual-Move instructions on the CFX. An equivalent pipelined copy on the CFX is given below:

```
// Copy NUM_WORDS from memory pointed to by xp0
// to memory pointed to by yp0
loop.2d (NUM_WORDS - 2), copy_loop_end
// Prime the copy loop in the delay slots
nop             ||| load a0, x[xp0++]
move.nx b0, a0 ||| load a0, x[xp0++]
copy_loop_end:
    move.nx b0, a0 ||| load a0, x[xp0++] !! store y[yp0++], b0
// Flush the copy pipeline
move.nx b0, a0 ||| store y[yp0++], b0
nop            ||| store y[yp0++], b0
```

5.3.11.2 Address Arithmetic and Addressing Modes

The CFX supports post-modification and circular addressing modes which are similar to the capability provided by the addressing modes of the RCore `PCFG` registers.

Each pointer register has an associated pair of registers, `step` and `circ`, which are used to specify the addressing mode. For example, the addressing mode associated with pointer register `xp0` is specified by the registers `xstep0` and `xcirc0`. The 16-bit `step` register contains the signed step-size value used in the pointer post-modification operation. The 15-bit `circ` register specifies the size (14-bit, unsigned) of the circular buffer.

In addition, a bit-reversed mode, intended to speed up algorithms such as the Fast Fourier Transform, is available. When a memory access is made using this mode, the pointer is bit-reversed (i.e., the bit at index 0 is moved to index 15, the bit at index 1 is moved to index 14, and so on), and the corresponding step register is bit-reversed. The two bit-reversed values are added, and the result is bit-reversed.

Simple post-modifications as well as post-modification using the `step` and `circ` registers are available for all indirect memory access instructions on the CFX. All regular Move instructions can use the following post-modifications:

- no modification
- -2, -1, +1, +2
- + `step` register
- - `step` register
- + `step` register modulo with `circ` register

Dual-Move instructions allow only a limited subset of post-modifications including the following:

- no modification
- +1
- + `step` register
- + `step` register modulo with `circ` register

The following example demonstrates some of the syntax required for indirect memory accesses with post-modification, as well as post-modification and circular addressing using the `step` and `circ` registers. This is compared to similar RCore assembly code which uses a `PCFG` register for pointer post-modification.

BelaSigna 250 and BelaSigna 200 code:

```
LD AH, (R0+)                            // post-modification of 1

// Set R0 addressing for buffer size of 2^5 and step size of 7
Config_Addr_Reg(0, 5, PCFG_POWER_OF_TWO, 7)
LD AH, (R0+!)                           // apply step-size, and wrapping if required
```

BelaSigna 300 code:

```
nop ||| load x0, x[xp0++]               // post-mod of 1
nop ||| load x0, x[xp0 -= 2]            // post-mod of -2

// Set xp0 addressing for buffer size of 2^5 and post-modification
// step-size of 7
Sys_CFX_CircAddrConfig(xcirc0, CIRC_CIRCULAR | 32)
Sys_CFX_StepConfig(xstep0, 7)
nop ||| load x0, x[xp0 +%= xstep0]      // apply post-mod and wrapping
nop ||| load x0, x[xp0 += xstep0]       // apply post-mod only
```

```
// Set xp0 addressing to bit-reversed mode and post-modification
// step-size of 8
Sys_CFX_CircAddrConfig(xcirc0, CIRC_BIT_REVERSE)
Sys_CFX_StepConfig(xstep0, 8)
```

5.3.12 Rounding and Saturation

Data movement from the accumulators on the CFX involves passing the accumulator data through a rounding and saturation unit. There are two bits in the status register (`SR_ROUND` and `SR_BIAS`) used to specify one of four possible rounding modes on the CFX when an accumulator is used as a source in a data-width conversion. The available rounding modes are:

- simple biased truncation
- sign-magnitude truncation
- simple biased rounding
- convergent rounding

Simple biased rounding is equivalent to the rounding performed by the `RND A` instruction on the RCore. More information regarding these modes is available in the *CFX DSP Architecture Manual*.

Note that because of the round/saturate units, rounding is done automatically on the CFX whereas on the RCore it required the explicit use of the `RND A` instruction.

Automatic saturation on the CFX is more closely analogous to the automatic limiting which occurs on the RCore when the `ST_OP` bit is set in the RCore's `ST` register. When saturation is enabled on the CFX using the `SR_SATURATE` bit, the value from the accumulator is automatically limited to the positive or negative full-scale value supported by the destination format.

The example that follows shows how the rounding, bias, and saturation modes are set and used on the CFX in comparison to the RCore.

BelaSigna 250 and BelaSigna 200 code:

```
SET ST, ST_OP              // set saturation mode
RND A                      // perform rounding
LD (R0+), AH               // perform saturation
```

BelaSigna 300 code:

```
Sys_CFX_SaturateEnable       // set saturation mode
Sys_CFX_RoundEnable          // set rounding mode
Sys_CFX_BiasEnable           // set simple bias mode (bit cleared)
nop ||| store x[xp0], a0     // perform rounding and saturation during data move
```

5.3.13 Interrupt Service Routines

At a high-level, the use of ISRs on the CFX is similar to their use on the RCore. A fixed location interrupt vector table is used to dispatch the processing of interrupts. However, the details of how the interrupts are handled by the core are substantially different for the CFX.

The CFX interrupt vector table entries are filled with unconditional branches to the start of the associated ISR (i.e., `goto.0d` instructions or the no-delay-slot synonym `goto`). On the RCore, the interrupt vector table is filled only with the addresses of the ISRs.

When an interrupt occurs, the CFX stores the program counter in the interrupt link program counter (`ilpc` register), stores the status register in the interrupt link status register (`ilsr` register) and disables interrupts. Since only one set of interrupt link registers exists, nested interrupts on the CFX are not supported by the hardware unless you explicitly store `ilpc` and `ilsr` on the stack. This contrasts with the RCore, which automatically stores the pre-interrupt `PC` on the system stack but does not store the status register `ST` (the ISR typically pushes `ST` onto the stack). Both cores allow arbitrary depth nested interrupts (up to the limits of the stack memory available).

When the ISR exits using a `returni` type instruction, the program counter and status register are restored from the `ilpc` and `ilsr` registers respectively. This contrasts with RCore ISRs, which typically restore the status register `ST` and enable interrupts prior to returning using `RET`.

5.4 FIFO CONTROLLER AND IOC CONFIGURATION

The FIFO controller and Input/Output Controller (IOC) provide similar, but more flexible, functionality to the Input/Output Processor (IOP) and Smart FIFOs on BelaSigna 250 and BelaSigna 200. Previous platforms had two fixed-size FIFO buffers (one input, one output) and limited ability to change the data flow between the IOP and the FIFOs. BelaSigna 300 allows you to specify your own FIFO buffers and to control the data flow from these buffers to system inputs and outputs via the IOC.

Information about configuring FIFOs for use with the WOLA Filterbank on BelaSigna 300 is included in Section 5.5.3, “Implementing the WOLA Filterbank”. More information on the FIFO controller and IOC in general can be found in the *Hardware Reference Manual for BelaSigna 300*. Specific examples of FIFO and IOC usage can be found in the BelaSigna 300 sample code.

5.4.1 FIFO Controller Configuration

The FIFO controller must be configured to activate the FIFOs necessary for a particular application. There are 8 FIFOs available in shared memory. The FIFOs A0, A1, A2, and A3 are located in A memory; B0, B1, B2, and B3 are located in B memory. Each active FIFO is defined in terms of the following properties, which are stored in FIFO configuration registers that are mapped to shared memory:

- Start and end addresses, specified by the `D_FIFO_x_START` and `D_FIFO_x_END` registers
- FIFO size, specified by the `D_FIFO_x_SIZE` register
- Block size, specified by the `D_FIFO_x_BLOCK_SIZE` register
- Base pointer, specified by the `D_FIFO_x_BASE_PTR` register
- Input/output block pointer, specified by the `D_FIFO_x_IOBLOCK_PTR` register

start and end addresses (also size)

Specify the area of memory used for the FIFO buffer. The size of the FIFO must also be set accordingly based on the start and end addresses. The start, end and size FIFO parameters are restricted to multiples of 32.

block size

Specifies the number of data words to be read from (or written to) the FIFO between interrupt events. This property is similar to the block size parameter associated with the IOP on BelaSigna 250 and BelaSigna 200.

base pointer

Specifies the address of the block of data that is mapped to the start address of the FIFO. This block of data typically corresponds to the newest block, thereby giving the CFX and HEAR a view of the FIFO that is similar to the Smart FIFO mapping on BelaSigna 250 and BelaSigna 200. The base pointer is updated automatically according to the block size and FIFO length when a new block of samples is read from (or written to) the FIFO.

input/output block (ioblock) pointer

Used to specify the address of the current block used by the IOC for one of the system's input or output signals. The ioblock pointer actually points to the top of this block of data, and the IOC reads or writes the data in descending address order from this pointer. This means that data in a FIFO is ordered such that older data is always at higher addresses.

If a FIFO is not used by the IOC for one of the system's inputs or outputs (i.e., the FIFO stores an internal signal), it is normally called a *software FIFO*, because the state of the FIFO is under the control of software on the CFX and the HEAR rather than the IOC hardware. When used as a software FIFO, the address specified by the ioblock pointer is used in FIFO accesses made through a special direct access register for that FIFO.

The base pointer and ioblock pointer are both updated automatically by the FIFO controller every time a block of samples is read or written via the IOC or the access register. A software FIFO's pointers can also be updated at any time by writing the value of the block size to the FIFO's block count register.

The following CFX code example configures FIFO A0 using the system firmware macro `Sys_FIFO_Configure`:

```
Sys_FIFO_Configure(SYS_FIFO_A0,        \
                   FIFO_A0_START,      \
                   FIFO_A0_LENGTH,     \
                   FIFO_A0_BLOCK_SIZE, \
                   FIFO_A0_BASE_PTR,   \
                   FIFO_A0_IOBLOCK_PTR)
```

Note that the HCT generates a macro called `HCT_FIFO_Configure` which contains the calls to `Sys_FIFO_Configure` for all of the application's FIFOs that have been configured via the HCT input file.

5.4.2 IOC Configuration

The IOC is significantly more configurable than the Input/Output Processor (IOP) in BelaSigna 250 and BelaSigna 200 applications. The IOC is more flexible in two respects:

- It has more options for handling different combinations of inputs and outputs.
- It is configurable in terms of which FIFOs it uses for reading or writing data.

The IOP from BelaSigna 250 and BelaSigna 200 was essentially fixed in terms of its FIFO usage and allowed only a few select input/output configurations based on the available system inputs and outputs.

The IOC configuration is split across three control registers as described below:

D_IOC_INPUT_CFG This register is used to specify the system inputs that the IOC routes to which FIFOs. Each system input (A/D converters or PCM) can be routed to any of the configured FIFOs.

D_IOC_FIFO_CFG This register is used to specify details about how the IOC stores and accesses data in each FIFO. It allows you to specify options for the data format used when reading or writing data to the FIFO.

D_IOC_OUTPUT_CFG This register is used to specify the system outputs that the IOC routes from which FIFOs. Each system output (digital direct drive or PCM) can be sourced from any of the configured FIFOs, or directly from any of the system inputs.

The IOC configuration allows for multiple inputs or outputs to be routed to or from the same FIFO. In these situations, the data is treated like a multi-channel interleaved data stream. For the case of two input or output channels, this is analogous to the stereo interleaved modes of the IOP from BelaSigna 250 and BelaSigna 200.

The following code example shows how to use the system firmware macros to set the IOC configuration registers, as compared to the configuration of the IOP on BelaSigna 250 and BelaSigna 200.

BelaSigna 250 and BelaSigna 200 code:

```
// Configure and start IOP
Set_Iop_Cfg(IOP_CFG_VAL)
```

BelaSigna 300 code:

```
// Configure how the IOC handles input data
Sys_IOC_InputConfig(IOC_INPUT_CFG_VAL)
// Configure how the IOC handles output data
Sys_IOC_OutputConfig(IOC_OUTPUT_CFG_VAL)
// Configure how the IOC interacts with the FIFOs
Sys_IOC_FIFOConfig(IOC_FIFO_CFG_VAL)
```

5.5 HEAR CONFIGURATION

This section provides a brief introduction to configuring the HEAR. Particular emphasis is placed on how to use the HEAR Configuration Tool (HCT) and the WOLA microcode module to create a microcode configuration that is equivalent to the WOLA filterbank coprocessor functionality on BelaSigna 250 and BelaSigna 200.

5.5.1 Application Design Considerations

The HEAR Configurable Accelerator provides significantly more advanced signal processing capabilities than the WOLA filterbank coprocessor available on BelaSigna 250 and BelaSigna 200. In addition to the WOLA Filterbank instructions, the HEAR can also perform many essential instructions that can be used as “building blocks” for signal processing algorithms commonly found in audio applications. Examples of these functions include the following:

- time-domain FIR filtering
- time-domain IIR filtering
- subband FIR filtering with complex-valued signals and coefficients
- vector multiplication, addition, subtraction

- rectangular-polar conversion
- mean, variance, correlation

The enhanced accelerator functionality allows for greater flexibility in how an application's processing blocks are distributed between the CFX and the HEAR. Thus, a high level of parallelism and power-efficiency is achievable through careful load balancing and algorithm design by the application developer.

As with previous platforms, shared memory is the primary mechanism of communication between the two cores. The amount of shared memory has been increased. Furthermore, access conflicts to the single-ported shared memory are handled at the hardware level, simplifying the task of programming applications that require concurrent accesses to shared memory.

The communication between the cores via interrupt signals has been enhanced. The CFX services 8 individual interrupt lines which correspond to the completion of HEAR functions, whereas only one interrupt is available for this purpose on BelaSigna 250 and BelaSigna 200 systems.

When using the HEAR Configurable Accelerator, it is often best to design the overall application around the major signal processing instructions to be performed by the HEAR. The shared memory and interrupts can then be used to facilitate synchronization and communication between the HEAR and the CFX.

Refer to the *HEAR Configurable Accelerator Reference Manual* for complete details of the functions supported by the HEAR. The BelaSigna 300 sample code also provides many examples of using the HEAR for enhanced signal processing tasks.

5.5.2 Configuration Using the HCT

The HCT is used to configure microcode modules for the HEAR Configurable Accelerator. The closest equivalent for BelaSigna 250 and BelaSigna 200 platforms is the macros provided by the `wola.inc` standard include file. The `wola.inc` include file provides macros which link in standard WOLA filterbank coprocessor microcode configurations. These available microcode configurations are pre-determined and pre-compiled into object code in the form of the `ucode.lib` and `windows.lib` standard libraries. With BelaSigna 300, the HCT now allows you to create a custom microcode configuration for your application.

Each custom microcode configuration is built from a set of available microcode modules. These modules encapsulate the signal processing instructions available on the HEAR. Each module has one or more functions. For example, the `WOLA` microcode module provides two functions (`WOLA_Analysis` and `WOLA_Synthesis`), which implement equivalent instructions to the WOLA filterbank coprocessor analysis and synthesis functions.

The input to the HCT is a source file (identified by the `.hct` extension). This source file is used to configure the HEAR microcode and the FIFO controller. The HCT syntax is a simple macro-based language where every statement is of the form:

```
macro(argument_list)
```

where `macro` is the name of a built-in macro or a microcode function, and `argument_list` is a comma-separated list of identifiers and/or expressions providing the parameters to `macro`.

After processing its source file input, the HCT generates an object code module (`.obj` file extension) containing the microcode configuration, and a CFX include file (`.cfi` file extension)

containing preprocessor definitions and macros that help your application to interface with the microcode configuration. The generated object module is automatically linked into your application as part of the IDE build process.

More information on using this tool can be found in the *HEAR Configurable Accelerator Reference Manual* or by consulting the porting example given below. The following section provides a more detailed example on porting a typical WOLA filterbank configuration from BelaSigna 250 or BelaSigna 200 to BelaSigna 300.

5.5.3 Implementing the WOLA Filterbank

The HEAR has the same functionality as the WOLA filterbank coprocessor for implementing a WOLA filterbank. By using the HCT, no statements are required within the CFX source files to configure the WOLA filterbank or other microcode features. This is entirely done using an HCT input file. This section describes how to use the HCT and the HEAR microcode to configure an BelaSigna 300 application that has WOLA filterbank functionality similar to that of an BelaSigna 250 or BelaSigna 200 application. It is helpful to read this section while consulting the BelaSigna 300 sample code for complete examples of an HCT source file.

Developing a microcode configuration which uses the HEAR's `WOLA` microcode module requires several steps:

1. Define constants to use in expressions or as parameters for configuring the module.
2. Configure an input and output FIFO for use with the `WOLA`.
3. Configure the `WOLA` module itself.
4. Assign the `WOLA` microcode module's functions to HEAR function chains.

5.5.3.1 Defining Constants

The HCT provides a simple mechanism for defining constants which you can use when configuring microcode modules. Conceptually, this ability replaces the use of `#defines` in application include files for configuring the WOLA filterbank coprocessor on BelaSigna 250 and BelaSigna 200. User-defined values and expressions can be assigned to user-defined identifiers using the `Define()` macro in the HCT source file. An example of using this macro to create constants for the WOLA parameters is given below:

```
Define(WOLA_La, 128)
Define(WOLA_Ls, 64)
Define(WOLA_N, 32)
Define(WOLA_R, 8)
Define(WOLA_STACKING, WOLA_STACKING_EVEN)
Define(WOLA_MODE, WOLA_MODE_MONO)
```

This example also illustrates the use of microcode constants. Each microcode module can define its own constants which can be used to help configure the microcode. The identifiers `WOLA_STACKING_EVEN` and `WOLA_MODE_MONO` are examples of these constants and they are defined by the `WOLA` microcode module.

The HCT also generates preprocessor definitions for each of these constants when generating its header file output. This simplifies the sharing of constant values between the HCT microcode configuration and the CFX application code. For example, the header file output for the HCT statements given above is as follows:

```
//
// Microcode Definitions
//
```

```
#define WOLA_La         (128)
#define WOLA_Ls         (64)
#define WOLA_N          (32)
#define WOLA_R          (8)
#define WOLA_STACKING   (WOLA_STACKING_EVEN)
#define WOLA_MODE       (WOLA_MODE_MONO)
```

5.5.3.2 Configuring the FIFO Controller

The HCT is normally used to assist with configuring the FIFO controller's hardware FIFO buffers. The FIFO configuration is often closely tied to the microcode configuration, so defining them in the same input file reduces the chance of misconfiguration.

The `FIFOConfigure()` macro instructs the HCT to generate code for configuring the FIFO controller using the given parameters. For the WOLA filterbank, we require an input FIFO and an output FIFO. These FIFOs correspond to the fixed input and output FIFOs available on BelaSigna 250 and BelaSigna 200. Configuration of the necessary FIFOs is analogous to proper configuration of the IOP on BelaSigna 250 and BelaSigna 200, however the increased flexibility of BelaSigna 300 means that additional options are available during this configuration.

The HCT statements given below configure FIFO A0 as an input FIFO for use with the `WOLA` module:

```
Define(FIFO_A0_BASE, 0)
Define(FIFO_A0_SIZE, 160)
Define(FIFO_A0_BLOCKSIZE, WOLA_R)
Define(FIFO_A0_BASE_PTR, 0)
Define(FIFO_A0_IOBLOCK_PTR, 0)

FIFOConfigure(FIFO_A0, FIFO_A0_BASE, FIFO_A0_SIZE, FIFO_A0_BLOCK_SIZE, FIFO_A0_BASE_PTR,
    FIFO_A0_IOBLOCK_PTR)
```

This results in FIFO A0 being configured for a block size of `WOLA_R`, and an overall size of 160 words. The FIFO size must be greater than `WOLA_La+WOLA_R` and must also be a multiple of 32 due to the configuration restrictions of the FIFO controller. For more information on the typical input FIFO configuration for the `WOLA` module, consult the *HEAR Configurable Accelerator Reference Manual*.

The HCT statements given below configures FIFO B0 as an output FIFO for use with the `WOLA` module:

```
Define(FIFO_B0_BASE, 0)
Define(FIFO_B0_SIZE, 96)
Define(FIFO_B0_BLOCKSIZE, WOLA_R)
Define(FIFO_B0_BASE_PTR, 0)
Define(FIFO_B0_IOBLOCK_PTR, WOLA_Ls + WOLA_R)

FIFOConfigure(FIFO_B0, FIFO_B0_BASE, FIFO_B0_SIZE, FIFO_B0_BLOCK_SIZE, FIFO_B0_BASE_PTR,
    FIFO_B0_IOBLOCK_PTR)
```

This results in FIFO B0 being configured for a block size of `WOLA_R`, and an overall size of 96 words. The FIFO size must be greater than `WOLA_Ls + WOLA_R` and must also be a multiple of 32 due to the configuration restrictions of the FIFO controller. For more information on the typical output FIFO configuration for the `WOLA` module, consult the *HEAR Configurable Accelerator Reference Manual*.

Using the `FIFOConfigure()` macro causes the HCT to generate preprocessor-based definitions and macros for configuring the associated FIFOs. This code is placed in the header file output by the HCT. Your CFX application can simply call the generated macros to configure the FIFOs and

also to reserve memory for the FIFOs. An example of these generated macros based on the above HCT input is given below:

```
//
// FIFO Configuration
//
#define HCT_FIFO_A0_START       (FIFO_A0_BASE)
#define HCT_FIFO_A0_LENGTH      (FIFO_A0_SIZE)
#define HCT_FIFO_A0_BASE_PTR    (FIFO_A0_BASE_PTR)
#define HCT_FIFO_A0_IOBLOCK_PTR (FIFO_A0_IOBLOCK_PTR)
#define HCT_FIFO_A0_BLOCK_SIZE  (WOLA_R)

#define HCT_FIFO_B0_START       (FIFO_B0_BASE)
#define HCT_FIFO_B0_LENGTH      (FIFO_B0_SIZE)
#define HCT_FIFO_B0_BASE_PTR    (FIFO_B0_BASE_PTR)
#define HCT_FIFO_B0_IOBLOCK_PTR (FIFO_B0_IOBLOCK_PTR)
#define HCT_FIFO_B0_BLOCK_SIZE  (WOLA_R)


//
// Configuration Macros
//
#macro HCT_FIFO_Configure
    Sys_FIFO_Configure(SYS_FIFO_A0,            \
                       HCT_FIFO_A0_START,      \
                       HCT_FIFO_A0_LENGTH,     \
                       HCT_FIFO_A0_BLOCK_SIZE, \
                       HCT_FIFO_A0_BASE_PTR,   \
                       HCT_FIFO_A0_IOBLOCK_PTR)

    Sys_FIFO_Configure(SYS_FIFO_B0,            \
                       HCT_FIFO_B0_START,      \
                       HCT_FIFO_B0_LENGTH,     \
                       HCT_FIFO_B0_BLOCK_SIZE, \
                       HCT_FIFO_B0_BASE_PTR,   \
                       HCT_FIFO_B0_IOBLOCK_PTR)
#endm

#macro HCT_FIFO_Reserve_Mem

    Sys_Mem_Start_Absolute_Segment(X_HCT_FIFO_A0_Memory_Reserved, \
                                   SYS_MEMSPACE_X, \
                                   (D_HEAR_AMEM_BASE + HCT_FIFO_A0_START))
    .bss HCT_FIFO_A0_LENGTH

    Sys_Mem_Start_Absolute_Segment(X_HCT_FIFO_B0_Memory_Reserved, \
                                   SYS_MEMSPACE_X, \
                                   (D_HEAR_BMEM_BASE + HCT_FIFO_B0_START))
    .bss HCT_FIFO_B0_LENGTH

#endm
```

The CFX application code normally calls the generated macro `HCT_FIFO_Configure` as part of the system initialization and normally calls the generated macro `HCT_FIFO_Reserve_Mem` as part of the application's static memory allocation. This example also illustrates a general concept for the HCT generated header file—all HCT-generated preprocessor definitions and macros are normally prefixed by the string `HCT_`. This can be used to distinguish between user-created code and HCT-generated code.

5.5.3.3 Configuring the WOLA Module

Before you can use the `WOLA` module's analysis and synthesis functions, an instance of the `WOLA` module must be properly configured. This is analogous to using the `WOLA_CONFIGURE()` macro from the BelaSigna 250 and BelaSigna 200 firmware. An example of configuring the WOLA microcode module is given below:

```
Define(ANALYSIS_FIFO_ADDR, D_HEAR_AMEM_BASE + FIFO_A0_BASE)
Define(SYNTHESIS_FIFO_ADDR, D_HEAR_BMEM_BASE + FIFO_B0_BASE)

Define(WOLA_RESULTS_OFFSET, 0x0)
Define(ANALYSIS_RESULTS_BASE, D_HEAR_H01MEM_BASE + WOLA_RESULTS_OFFSET)

WOLA_Configure(WOLA_A, WOLA_La, WOLA_Ls, WOLA_N, WOLA_R,
               WOLA_STACKING, WOLA_MODE,
               WOLA_WIN_MODE_DEFAULT, WOLA_WIN_MODE_DEFAULT,
               ANALYSIS_FIFO_ADDR, SYNTHESIS_FIFO_ADDR,
               ANALYSIS_RESULTS_BASE,
               ANALYSIS_RESULTS_BASE,
               D_HEAR_H01MEM_BASE + WOLA_RESULTS_OFFSET,
               D_HEAR_H23MEM_BASE + WOLA_RESULTS_OFFSET)
```

The `WOLA_Configure()` microcode function causes the HCT to generate an instance of the `WOLA` microcode module. The first parameter to this function is a unique, user-supplied name for this instance. This is required because the HEAR microcode allows for multiple distinct instances of each microcode module. The remaining parameters provide the configuration and memory addresses needed by the WOLA module. For detailed information on these parameters, consult the *HEAR Configurable Accelerator Reference Manual*.

The idea of specifying the basic WOLA filterbank parameters such as L, N, R, stacking, channel mode, etc. is the same for the HEAR microcode as it is for using the `WOLA_CONFIGURE()` macro with BelaSigna 250 and BelaSigna 200. The ability to specify the addresses of inputs and outputs such as the input FIFO, output FIFO, analysis results, etc. is new in the HEAR microcode, and represents a substantial increase in flexibility for microcode configurations. Most parameters also have some restrictions on the valid range of values they can take. These restrictions are documented in the *HEAR Configurable Accelerator Reference Manual* and are also checked automatically when the HCT generates the microcode configuration.

5.5.3.4 Analysis, Gain and Synthesis Functions

A standard WOLA filterbank coprocessor microcode configuration for BelaSigna 250 and BelaSigna 200 involves analysis, gain and synthesis function. These same functions are available as part of the HEAR microcode, but you must assign them to specific function chains as part of the application's microcode configuration. An example of assigning the analysis and synthesis functions to separate function chains using the HCT is given below:

```
StartFunctionChain(CMD0, analysis_function_chain)
    // Start the analysis function
    WOLA_Analysis(WOLA_A)

    // Trigger the HEAR_0 interrupt on the CFX
    Event(0)
EndFunctionChain()

StartFunctionChain(CMD1, synthesis_function_chain)
    // Start the synthesis function
    WOLA_Synthesis(WOLA_A)

    // Trigger the HEAR_1 interrupt on the CFX
    Event(1)
EndFunctionChain()
```

Each function chain is delineated by using the `StartFunctionChain` and `EndFunctionChain` macros. The `StartFunctionChain` macro requires you to specify the hardware function chain to be used (`CMD0` and `CMD1` in this case) as well as a unique user-supplied name for the function chain. Microcode functions in between these two built-in macros are assigned to that function chain and execute in the order which they appear. In these function chains, the `WOLA_Analysis` and `WOLA_Synthesis` microcode functions reference the already configured `WOLA_A` instance of

the WOLA module. The built-in Event microcode function triggers one of the interrupts on the CFX which are connected to the HEAR (see Section 5.5.3.5, “CFX-HEAR Communication”).

For the gain application function, the HEAR microcode provides separate real and complex-valued gain application microcode modules. These modules are separated from the WOLA module, thereby allowing you to specify an arbitrary number and type of gain applications if desired. When emulating the functionality of BelaSigna 250 and BelaSigna 200, we can configure and use the real gain application module (named the Gain_R module) as part of its own function chain. An example of this is given below:

```
Define(GAIN_VECTOR_OFFSET, 0x0)
Define(GAIN_TABLE_BASE, D_HEAR_H45MEM_BASE + GAIN_VECTOR_OFFSET)

StartFunctionChain(CMD2, gain_function_chain)
    // Start the gain application function
    Gain_R(GAIN_A, WOLA_N, WOLA_STACKING, WOLA_MODE,
           ANALYSIS_RESULTS_BASE,
           GAIN_TABLE_BASE,
           ANALYSIS_RESULTS_BASE)

    // Trigger the HEAR_2 interrupt on the CFX
    Event(2)
EndFunctionChain()
```

This example illustrates another key microcode concept. One obvious difference between the WOLA module and the Gain_R module is that the WOLA module requires the use of a separate configuration function (WOLA_Configure), whereas the configuration and execution of the Gain_R function is handled with one microcode function. This is because the WOLA module involves multiple functions (specifically analysis and synthesis) which share the same set of parameters. The WOLA_Configure function is used to set these common parameters. For a microcode module with only one function (like Gain_R), it is possible to specify all of the parameters when the microcode function is actually used in a function chain.

Given the input above, the HCT generates the microcode for the three function chains specified. It also generates preprocessor definitions in the header file output which can be used to configure the HEAR’s function chain controller and trigger the function chains from the CFX application code. An example of this output from the function chains above is as follows:

```
//
// Function Chain Controller Configuration
//
#define analysis_function_chain  HEAR_FC_CMD0_POS
#define synthesis_function_chain HEAR_FC_CMD1_POS
#define gain_function_chain      HEAR_FC_CMD2_POS

#define HCT_HEAR_FC_ENABLE_VAL (HEAR_FC_EBL_CMD0 | \
                                HEAR_FC_EBL_CMD1 | \
                                HEAR_FC_EBL_CMD2)
#define HCT_HEAR_FC_PRIORITY_VAL (0)
```

5.5.3.5 CFX-HEAR Communication

The final step in creating and using a WOLA filterbank microcode configuration with the HEAR is triggering the function chains from the CFX application code and writing CFX ISRs to respond to the signals coming back from the HEAR. This is analogous to starting a WOLA filterbank coprocessor function and writing an ISR for the WOLA_DONE interrupt on BelaSigna 250 or BelaSigna 200.

The system firmware provides a macro for starting a function chain from the CFX: `Sys_HEAR_Command`. This macro can be used with the HCT generated preprocessor definitions for the function chains as follows:

```
// Start analysis function chain
Sys_HEAR_Command(analysis_function_chain)

. . .

// Start gain function chain
Sys_HEAR_Command(gain_function_chain)

. . .

// Start synthesis function chain
Sys_HEAR_Command(synthesis_function_chain)
```

With BelaSigna 300, it is also possible to define function chains that are triggered directly by the interrupts associated with each FIFO. These FIFO related functions chains are able to start automatically when the FIFO interrupt occurs, thereby eliminating the need for the CFX to issue a command.

Interrupts triggered from the microcode function chains are used to alert the CFX when the function chains have reached a particular point in their execution. Eight of these interrupts are available to you. In this WOLA filterbank example, this is done using the `Event` microcode function at any point in a function chain. Your CFX code can define ISRs to respond to each of these interrupts. Note that BelaSigna 250 and BelaSigna 200 have only one interrupt to signal the completion of a function on the WOLA filterbank coprocessor. The availability of multiple interrupts between the HEAR and the CFX can greatly simplify the application code on BelaSigna 300.

5.6 OTHER SYSTEM CONFIGURATION TASKS

This section discusses some other examples of porting functionality from BelaSigna 250 and BelaSigna 200 to BelaSigna 300. These generally relate to using the peripherals and analog blocks provided by BelaSigna 300.

5.6.1 Configuration of Analog Blocks

As with previous platforms, an important part of the initialization portion of an application on BelaSigna 300 relates to the setup of the system's analog blocks. This includes components such as the input stage, output stage, and power supply.

Configuration of the analog blocks is normally done using the macros and definitions provided by the system firmware. As on BelaSigna 250 and BelaSigna 200, the analog configuration registers for BelaSigna 300 are always named using the `A_` prefix. The BelaSigna 300 system firmware provides explicit “set” macros for each analog configuration register.

The typical set of macros used for analog block configuration in a BelaSigna 250 or BelaSigna 200 application is given below:

```
Write_AReg(A_ADC_CTRL,      ADC_CTRL_DEFAULT)     // ADC
Write_AReg(A_DAC_CTRL,      DAC_CTRL_DEFAULT)     // DAC
Write_AReg(A_INPUT_CTRL,    INPUT_DEFAULT)        // Input stage
Write_AReg(A_INFILT_CTRL,   INFILT_CTRL_DEFAULT) // Input filter config
Write_AReg(A_OUTPUT_CTRL,   O_CTRL_DEFAULT)       // Output stage
Write_AReg(A_IN_GAIN_CTRL,  IN_GAIN_DEFAULT)      // Input gain
Write_AReg(A_OUT_ATTN_CTRL, OUT_ATT_DEFAULT)      // Output attenuation
```

```
Write_AReg(A_ADC_GF0_CTRL,  GF0_DEFAULT)         // Decimation gain (chan. 0)
Write_AReg(A_ADC_GF1_CTRL,  GF1_DEFAULT)         // Decimation gain (chan. 1)
Write_AReg(A_DEL_INT_CTRL,  FRACTIONAL_INT)      // Integer part of delay
Write_AReg(A_DEL_FRAC_CTRL, FRACTIONAL_FRAC)     // Fractional part of delay
Write_AReg(A_PSU_CTRL,      PSU_CTRL_DEFAULT)    // Power supply
```

The roughly equivalent set of macros for BelaSigna 300 is as follows:

```
Sys_Set_ADC_CFG(ADC0_DC_REMOVE_BYPASS | LPF0_IN_BYPASS)
Sys_Set_O_DAC_CTRL(O_DAC_CTRL_VAL)
Sys_Set_INPUT_CTRL(INPUT_CTRL_VAL)
Sys_Set_OUTPUT_CTRL(OUTPUT_CTRL_VAL)
Sys_Set_IN_GAIN_CTRL(IN_GAIN_CTRL_VAL)
Sys_Set_FRAC_DELAY_0_CTRL(FRAC_DELAY_0_CTRL_VAL)
```

The *Hardware Reference Manual for BelaSigna 300* contains details about the settings available for the analog control blocks. In addition, the sample applications that are available as part of the EDK contain examples of typical settings.

5.6.2 Interrupt Controller Configuration

The interrupt configuration section of an application's initialization generally consists of the following steps:

- configure priority settings
- enable individual interrupts (un-mask interrupts)
- clear pending interrupts
- enable interrupts globally

As on BelaSigna 250 and BelaSigna 200, the BelaSigna 300 system firmware provides macros to set the registers associated with these settings. For BelaSigna 250 and BelaSigna 200, the `D_INT_CTRL`, `D_INT_STATUS`, `D_INT_EBL`, and status registers are updated with the following RCore code:

```
// Set interrupt configuration: auto acknowledge and default priority
// Modifies D_INT_CTRL
Set_Int_Config(INT_AUTO_ACK_ENABLE | INT_PRIORITY_ENABLE, INT_PRIO_DEFAULT)

// Set interrupt enable
// Modifies D_INT_EBL
Set_Int_Enable(INT_ENABLE_VAL)

// Clear any interrupt requests that might be pending
// Modifies D_INT_STATUS
Interrupt_Ack(INT_ACK_ALL_INTS)

// Enable interrupts
// Modifies ST register
Enable_Int
```

Macros with the corresponding functionality are available for the CFX, as shown in the following example.

```
// Configure the interrupt priorities
// Modifies D_INT_PRIORITY
Sys_Int_Priority(INT_PRIORITY_VAL)

// Enable interrupts
// Modifies D_INT_ENABLE
Sys_Int_Enable(INT_ENABLE_VAL)

// Clear any pending interrupts
// Modifies D_INT_STATUS
```

```
Sys_Int_Ack(INT_STATUS_VAL)

// CFX-level interrupt enable
// Modifies sr register
Sys_CFX_IntEnable
```

Interrupts are enabled globally through the D_INT_ENABLE register of the interrupt controller, and through the status register of the CFX. Set both registers for proper initialization of the application. For the interrupt controller, use the INT_EBL_MASTER definition when setting the D_INT_ENABLE register. For the CFX, use the Sys_CFX_IntEnable macro to set the sr.ie bit of the status register to enable interrupt handling on the CFX. In the CFX example above, INT_ENABLE_VAL is defined to contain INT_EBL_MASTER.

5.6.3 Clock Configuration

The system's clocks are configured by the A_CLK_CTRL, D_CLK_CFG0, and D_CLK_CFG1 registers. A_CLK_CTRL controls the oscillator and determines the system clock rate, while D_CLK_CFG0, and D_CLK_CFG1 configure the various options for external clocking and clock prescaling.

The following example shows how the Sys_Set_CLK_CTRL and Sys_Clk_Config macros are used to write to these registers, as compared to a typical clocking configuration for BelaSigna 250 or BelaSigna 200.

BelaSigna 250 and BelaSigna 200 code:

```
// Disable internal oscillator
#define CLK_CTRL_CFG        SYS_CLK_OFF
// Enable external clock
#define EXTCLK_SELECT_VAL   CLK_EXTCLK_IN_ENABLE
// Disable USRCLK
#define UCLK_SELECT_VAL     CLK_UCLK_DISABLE
// PCLK configuration
#define PCLK_PRESCALE_VAL   CLK_PCLK_PRESCALE_2
// MCLK configuration
#define MCLK_PRESCALE_VAL   CLK_MCLK_PRESCALE_2
// USRCLK configuration (needs to be defined even when USRCLK disabled)
#define UCLK_PRESCALE_VAL   CLK_USRCLK_PRESCALE_1

// Set D_CLKSEL_CFG
Set_Clock_Select(EXTCLK_SELECT_VAL, UCLK_SELECT_VAL)
// Set A_CLK_CTRL register
Write_AReg(A_CLK_CTRL, CLK_CTRL_CFG)
// Set D_CLKDIV_CFG and D_USRCLKDIV_CFG
Set_Clock_Prescalers(PCLK_PRESCALE_VAL, MCLK_PRESCALE_VAL, UCLK_PRESCALE_VAL)
```

BelaSigna 300 code:

```
// Disable the internal oscillator
#define CLK_CTRL_VAL                (SYSCLK_OFF)
// Select the primary oscillator,
// enable EXTCLK, set the HEARCLK prescale to 1
// and set the EXTCLK prescale to 1.
// This application expects a 38.4 MHz external clock.
#define CLOCKCONFIG0_VAL            (SYSCLK_SEL_DEFAULT     | \
                                     EXTCLK_IN_ENABLE       | \
                                     HEARCLK_PRESCALE_1_000 | \
                                     EXTCLK_PRESCALE_1)
// Set the SLOWCLK prescale value to 30
// such that SLOWCLK runs at 1.28MHz,
// disable system clock double access mode
// and set the UCLK prescale value to 1.
// Configure PCM clocking with the default settings.
#define CLOCKCONFIG1_VAL            (DBL_ACCESS_MODE_DISABLE    | \
                                     SLOWCLK_PRESCALE_30        | \
```

```
                                        UCLK_PRESCALE_1             | \
                                        PCM_CLK_PAD_AS_INPUT        | \
                                        PCM_FRAME_PAD_SEL_DIS)
// Set D_CLK_CFG0 and D_CLK_CFG1 registers
Sys_Clk_Config(CLOCKCONFIG0_VAL, CLOCKCONFIG1_VAL)
// Set A_CLK_CTRL register
Sys_Set_CLK_CTRL(CLK_CTRL_VAL)
```

Refer to the *Hardware Reference Manual for BelaSigna 300* for information on the oscillator and clocking options.

5.6.4 Shared Memory Access

The shared memory is mapped to X memory and can be accessed by the CFX and HEAR. The IOC can also access a limited subset of the shared memory through the FIFO controller. Access to shared memory is coordinated by the shared memory bus controller. On BelaSigna 250 and BelaSigna 200, dual-ported memories and fixed memory access priorities are used to facilitate shared access.

When a memory conflict occurs due to the attempted access to the same memory instance by both the CFX and HEAR on the same cycle, the shared memory bus controller resolves this conflict according to access priority settings. The unit with the higher priority (CFX or HEAR) is granted access to the memory, and the other has its execution stalled until the higher-priority access is complete.

Due to the hardware support for resolving memory access conflicts, there is little burden on the software to ensure safe access to shared memory. You simply have to choose the priority setting and account for the effect of stalled memory accesses on the timing of the application. Despite this transparent shared access, it is still preferable to structure the application to minimize the chances of memory access conflicts since they can lead to non-deterministic application timing.

The following example shows how the priority for shared memory accesses is set.

```
// Set shared memory access priority for HEAR
Sys_Mem_Priority(SYSTEM_PRIORITY_HEAR)

// Set shared memory access priority for CFX
Sys_Mem_Priority(SYSTEM_PRIORITY_CFX)
```

The details of the shared memory mapping and the shared memory bus controller are available in the *Hardware Reference Manual for BelaSigna 300*.

5.6.5 Low-power Mode

The `sleep` instruction of the CFX is similar to the `sleep` instruction of the RCore. However, a CFX instruction allows for an optional address which specifies the program branch address to be used when the core restarts.

The following example shows how the `sleep` instruction is used.

BelaSigna 250 and BelaSigna 200 code:

```
// Stop the RCore, then restart when an interrupt occurs
SLEEP IE
```

BelaSigna 300 code:

```
// Stop the CFX, then continue to next instruction when
// an interrupt occurs
sleep

// Stop the CFX, then go to RESTART_ADDR when an interrupt occurs
sleep RESTART_ADDR
```

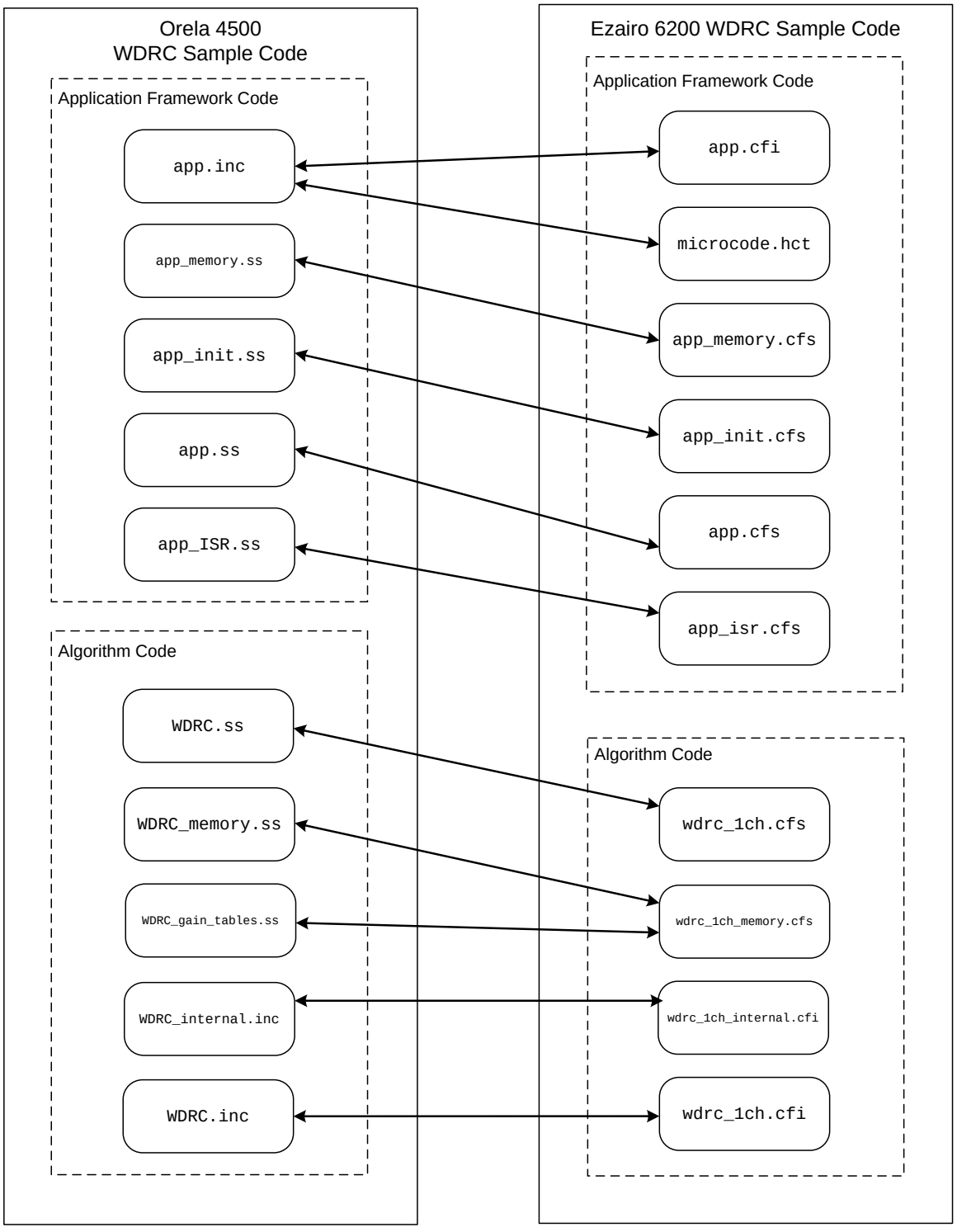
5.7 SAMPLE CODE STRUCTURE COMPARISON

This section illustrates some of the porting concepts described in the previous sections by comparing an BelaSigna 250 sample code application with its BelaSigna 300 version. For comparison, the Single-Channel Wide Dynamic Range Compression (WDRC) samples from both platforms are compared.

The BelaSigna 300 and BelaSigna 250 sample code has been implemented using a common source file and code structure to improve ease of use, simplify source code maintenance, and facilitate application modifications. You are certainly free to develop your own structures that suit your needs. In this section, however, for the purpose of illustration, the structure implemented in the sample code is assumed.

5.7.1 Source File and Code Structure

Since the source file and code structures of the BelaSigna 300 and BelaSigna 250 sample code applications are similar, it is straightforward to make comparisons between the two samples. Figure 7 presents a comparison of the source file organizations.


Orela 4500
WDRC Sample Code
Application Framework Code
app.inc
app_memory.ss
app_init.ss
app.ss
app_ISR.ss
Algorithm Code
WDRC.ss
WDRC_memory.ss
WDRC_gain_tables.ss
WDRC_internal.inc
WDRC.inc
Ezairo 6200 WDRC Sample Code
Application Framework Code
app.cfi
microcode.hct
app_memory.cfs
app_init.cfs
app.cfs
app_isr.cfs
Algorithm Code
wdrc_1ch.cfs
wdrc_1ch_memory.cfs
wdrc_1ch_internal.cfi
wdrc_1ch.cfi

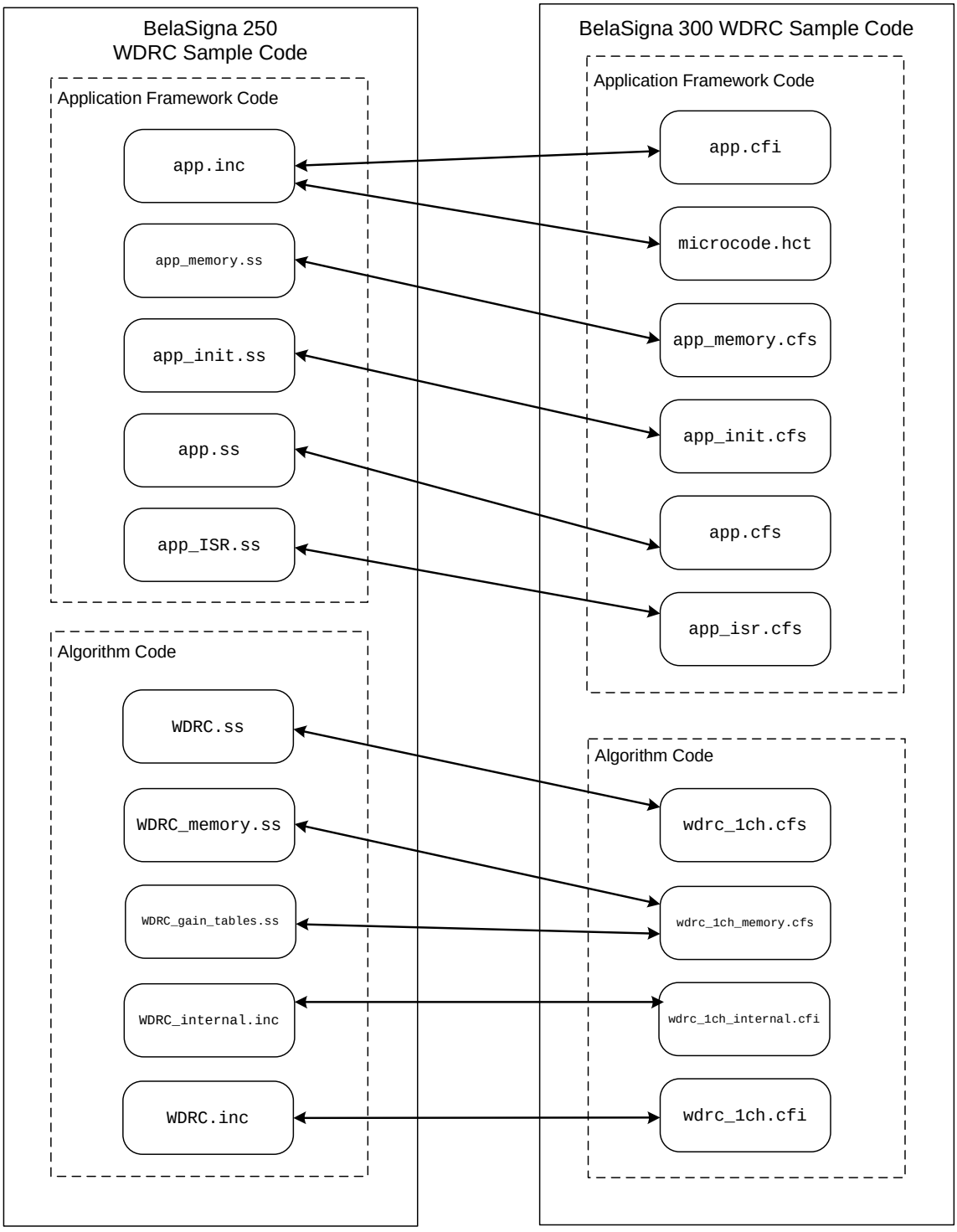


Figure 7: Comparison of Sample Code File Organizations

In Figure 7, an arrow indicates a functional correspondence between the content of two source files. For example, initialization code can be found in the file app_init.ss for the BelaSigna 250 version of the sample code; however, similar initialization code is in the file app_init.cfs for the BelaSigna 300 version of the sample code. In the case where one source file is linked to multiple others, this means that the functional correspondence is split among different files. For example, in the BelaSigna 250 version of the sample code, the file app.inc contains all of the configuration data for the sample. However, in the BelaSigna 300 version the corresponding configuration data and assembler directives are split among multiple source files: app.cfi and microcode.hct, as indicated in Figure 7. Thus, this figure helps to direct you to the correct source files when comparing the source code in the different versions of the sample.

5.7.2 File-by-file Comparison

In this section, each source file in the BelaSigna 250 version of the sample code is compared with the corresponding source file in the BelaSigna 300 version, as indicated by Figure 7 on page 66. The focus is on a high-level, functional comparison instead of a code-level or syntax-level comparison, which can be found in the preceding sections of this chapter.

app.inc

The source file app.inc contains the definitions for preprocessor symbols used to configure the system, preprocessor symbols for named constants and .extern directives for allocated memory, application functions and other assembler symbols. For BelaSigna 300, the preprocessor symbols for hardware configuration values and named constants are defined in app.cfi. This file also contains .declare directives for allocated memory; these directives correspond to the .extern directives in app.inc. The BelaSigna 300 equivalents of some preprocessor symbols defined in app.inc are also now part of the microcode configuration which is found in the microcode.hct source file (e.g., the WOLA filterbank configuration).

app_memory.ss

The source file app_memory.ss contains segment declarations and symbols used to allocate application framework memory using the assembler and linker. A number of system macros are used to create data segments and allocate named locations in these segments for variables, buffers and tables. The app_memory.cfs source file in the BelaSigna 300 sample code contains similar kinds of segment declarations and memory allocation using the BelaSigna 300 firmware macros.

app_init.ss

The source file app_init.ss contains the source code for the application framework initialization function. In the BelaSigna 300 version of the sample code, the initialization function can be found in the file app_init.cfs. In both cases, the initialization functions perform hardware configuration. In the BelaSigna 250 sample code, the initialization function also dynamically initializes variables in memory. In the BelaSigna 300 sample code, variable initialization is done using statically initialized memory in app_memory.cfs and wdrc_ch1_memory.cfs.

app.ss

The source file app.ss contains the main application framework code of the sample. The framework code includes function calls to the initialization functions, and the main loop structure. For the BelaSigna 300 sample, the corresponding code can be found in the file app.cfs.

app_ISR.ss

The source file `app_ISR.ss` contains the source code for the interrupt service routines which are part of the application framework. For the BelaSigna 300 sample, the corresponding code can be found in the file `app_isr.cfs`.

WDRC.ss

The source file `WDRC.ss` contains the bulk of the source code that implements the WDRC function, such as the source code for calculating the input signal levels, and for calculating the compression gains. In the BelaSigna 300 version of the sample code, the source code that implements the WDRC function is in the file `wdrc_1ch.cfs`.

Note that while both the BelaSigna 300 version and the BelaSigna 250 version of the sample code implement a single-channel WDRC algorithm, the way the algorithm has been implemented in each version is not exactly the same. Thus, the content of the source files corresponds to each other only in a broad functional sense, and not in a strict line-by-line sense.

WDRC_memory.ss

The source file `WDRC_memory.ss` contains segment declarations and symbols used to allocate algorithm memory using the assembler and linker. A number of system macros are used to create data segments and allocate named locations in these segments for variables, buffers and tables. The `wdrc_1ch_memory.cfs` source file in the BelaSigna 300 sample code contains similar kinds of segment declarations and memory allocation using the BelaSigna 300 firmware macros.

WDRC_gain_tables.ss

The source file `WDRC_gain_tables.ss` specifies the default gain tables for frequency shaping and dynamic range compression in the BelaSigna 250 sample code. Similar gain tables can be found in the file `wdrc_1ch_memory.cfs` for the BelaSigna 300 sample code.

WDRC_internal.inc

The source file `WDRC_internal.inc` contains preprocessor definitions of named constants and `.extern` directives for symbols which are considered “internal” to the WDRC algorithm. That is, these definitions and directives are for constants, variables or functions which do not need to be accessed by the application framework. In the BelaSigna 300 version of the sample code, these definitions and directives can be found in the files `wdrc_1ch_internal.cfi`.

WDRC.inc

The source file `WDRC.inc` contains preprocessor definitions of named constants and `.extern` directives for symbols which are considered “external” to the WDRC algorithm. That is, these definitions and directives are used by the application framework to interface with the WDRC algorithm by accessing variables or calling functions which are defined in the WDRC code. In the BelaSigna 300 version of the sample code, these definitions and directives can be found in the files `wdrc_1ch.cfi`.

A Glossary

The following abbreviations and terms are used in this manual:

algorithm partitioning
: Separating your algorithm to assign different parts of it to run on multiple processors.

CFX Digital Signal Processor
: the BelaSigna 300 processing core that is used to configure the system and coordinate the flow of signal data progressing through the system

data sink
: Something that reads data from a FIFO.

data source
: Something that writes data into a FIFO.

delay slot
: A technique used to reduce the effect of pipeline breaks by executing the instruction (in the delay slot) that is after a control flow instruction before performing the control flow action.

FIFO
: A section of memory that is treated as a circular buffer where components that use this memory use pointers to handle data in a first-in, first-out manner. A *hardware FIFO* is a FIFO instance where its registers are controlled by the IOC. A *software FIFO* is a FIFO instance where its registers are controlled by an application on the CFX or microcode configuration on the HEAR.

function chain
: A sequence of microcode functions that are executed on the HEAR Configurable Accelerator in response to a command from the CFX or an event from the FIFO controller.

GPIO
: general purpose input/output

HEAR Configurable Accelerator
: the BelaSigna 300 core that is a microcode configurable signal processing engine used to execute a variety of signal processing functions; designed to work with the CFX to reduce the signal processing load on the CFX

I^2C
: Industry standardized Inter-IC Communication protocol. Support for a limited subset of this communication interface was called *TWSS* in previous EDKs.

I^2S
Industry standardized Inter-IC Sound protocol

IOC
input/output controller

ISR
interrupt service routine

LSAD
low-speed analog-to-digital

NVM
non-volatile memory

PCM
pulse code modulation

RCore Digital Signal Processor
The general-purpose programmable core for BelaSigna 250 and BelaSigna 200.

SPI
serial peripheral interface

time-slicing
A technique for partitioning your algorithm over the time dimension to free up clock cycles for other more critical computations.

TWSS
two wire synchronous serial interface

UART
universal asynchronous receiver-transmitter

WDRC
Wide Dynamic Range Compression

WOLA filterbank coprocessor
Weighted Overlap-Add filterbank coprocessor; a dedicated, configurable processor that performs filterbank analysis, synthesis and other vector-based computations for BelaSigna 250 and BelaSigna 200.

This page left blank for printing purposes.

Global Headquarters
ON Semiconductor
5005 East McDowell Road
Phoenix, Arizona 85008 USA
Tel: +1 602 244 6600

Website
www.onsemi.com

Copyright © 2009 Semiconductor Components Industries, LLC d/b/a ON Semiconductor. ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA and SIGNAKLARA are either trademarks or registered trademarks of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.